most-exchange

Operator's Manual

Installing, configuring and operating a deterministic low-latency matching engine: the shard and its five processes, the control plane that authors what they boot from, and the interfaces an operator drives them through.

Revision 2026-11-01

Applies to the current implementation, at roughly 80% of the specified feature set.
Sections marked NOT IMPLEMENTED describe behaviour that is designed but not built.

© 2026 P.M.Vrolijk. Licensed under CC BY 4.0: https://creativecommons.org/licenses/by/4.0/

Contents

1 **Introduction**

- 1.1 What this system is
- 1.2 The processes
- 1.3 Two boundaries, both binary
- 1.4 The two kinds of configuration
- 1.5 Prices, quantities and identifiers
- 1.6 Session phases
- 1.7 What is not built
- 1.8 How to read this manual

2 **Development deployment and operation**

- 2.1 Prerequisites
- 2.2 Build
- 2.3 The whole stack at once
- 2.4 Verifying the stack came up
- 2.5 The operator CLI
- 2.6 Driving a trade
- 2.7 Watching the book
- 2.8 The admin UI
- 2.9 What is development-only
- 2.10 Running without Docker
- 2.11 Automated end-to-end checks
- 2.12 Shutting down

3 **Production deployment**

- 3.1 The rule that shapes everything
- 3.2 Topology
- 3.3 Machine preparation
- 3.4 Core allocation
- 3.5 Aeron buffer sizing
- 3.6 Cluster configuration
- 3.7 Market data: multicast
- 3.8 Process management
- 3.9 Bringing a shard up
- 3.10 Blocking gaps

4 **Configuration in full**

- 4.1 How a process is configured
- 4.2 The shard security file
- 4.3 The participant registry

4.4 Engine
4.5 Gateway
4.6 Market data
4.7 Discovery
4.8 The cluster host
4.9 The control plane
4.10 Authoring configuration in the control plane
4.11 Publishing a release

5 **Daily operations, order flow and monitoring**
5.1 The trading day
5.2 The trading calendar
5.3 Operator commands
5.4 Order flow
5.5 Market data operations
5.6 Snapshots and restarts
5.7 Measuring
5.8 Monitoring

6 **Troubleshooting and maintenance**
6.1 First moves
6.2 Startup failures
6.3 The engine refuses to restore its snapshot
6.4 Recovering a halted security
6.5 Order flow problems
6.6 Market data problems
6.7 Maintenance procedures
6.8 Reading the logs
6.9 Consolidated list of what is not implemented

7 **Index**
7.1 Terms and concepts
7.2 Commands
7.3 Configuration keys and environment variables
7.4 Further reading

1. Introduction

1.1 What this system is

most-exchange is a matching engine for continuously traded, order-driven markets. It runs as one or more **shards**; each shard hosts up to ten securities, holds an independent limit order book for each of them, and is replicated across an odd number of machines by Aeron Cluster's Raft implementation.

A shard is a **deterministic single-threaded state machine**. One thread consumes a replicated log of commands and applies them in order; every node runs the identical code over the identical log and must arrive at byte-identical state. Everything an operator can do to a running market is expressed as a message in that log, and nothing an operator does is applied on one node and not another.

The design targets 100,000 orders per second per security across ten securities on a shard — a one-microsecond aggregate budget per order. The engine's own share of that, measured on a development machine, is 0.42 µs at the median for a whole message.

> **SHARD**
> The unit of replication and of scale. One shard is one Raft group, one replicated log, one matching thread, and at most ten securities with fully independent books. There is no cross-instrument matching, so capacity is added by running more shards, never by adding securities to one.

> **DETERMINISM**
> The property that the same log produces the same state on every node. It is why wall-clock reads, unbounded allocation and unvalidated array indexing are prohibited on the engine's path: a crash or a divergence does not affect one node, it affects every node at the same log position simultaneously.

1.2 The processes

A shard is five processes plus a control plane. They are separate processes on purpose — the media driver allocates memory and the engine must not, market data fan-out must never touch the matching thread, and the gateway is restarted on a different schedule from the Raft group.

Process	Binary	Role
Cluster host	`most cluster`	Aeron media driver, archive and consensus module in one process. Owns the replicated log, and authenticates connecting gateways.
Engine	`engine`	The Aeron `ClusteredService` — books, matching, auctions, snapshots. Attaches to a consensus module that must already exist.
Gateway	`gateway`	Order entry. Validates `securityId`, forwards to the cluster, translates execution reports back out. Holds no per-order state.
Market data	`market-data`	Consumes the engine's book event stream and derives L1, L2 and L3 plus an L2 recovery snapshot feed.
Discovery	`discovery`	Broadcasts the tradable universe — symbol to shard to gateway endpoints — on a repeating cycle.

Process	Binary	Role
Control plane	`control`	Authors reference data in Postgres, publishes immutable release artifacts, drives operator commands, runs the trading calendar, and serves the admin UI.

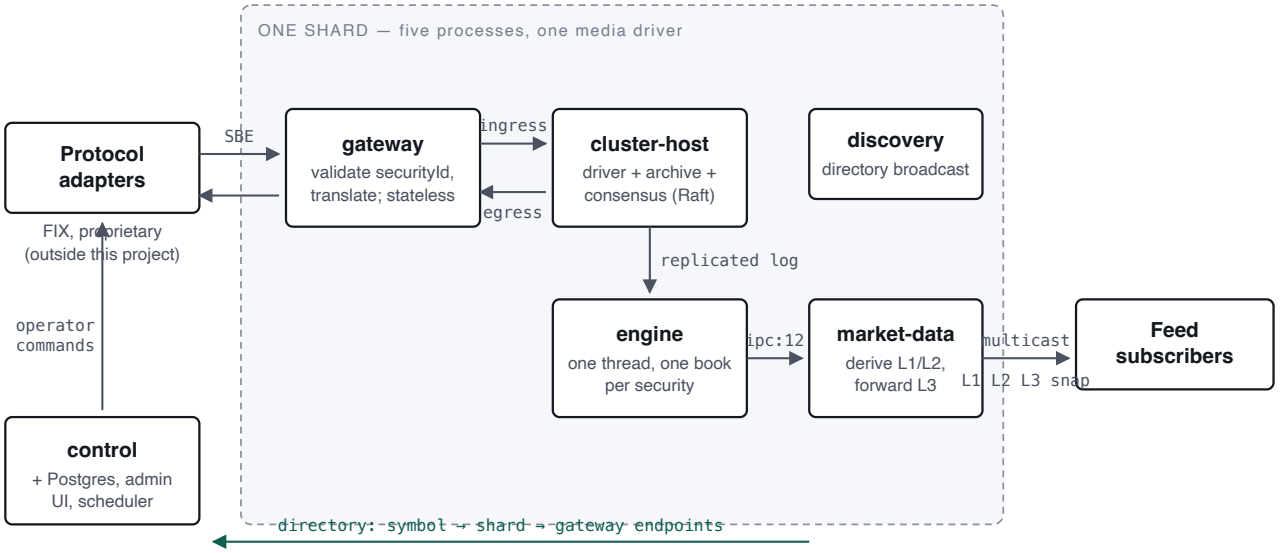


Figure 1.1 The five shard processes, the control plane, and what flows between them.

Only the control plane is authenticated, and the reason is worth stating plainly: it is the only process that *decides* something. The other five apply a replicated log or forward bytes.

1.3 Two boundaries, both binary

Order entry and market data are both **SBE over Aeron**. There is no FIX anywhere in this project. Protocol adapters that speak FIX or a proprietary session protocol sit upstream of the gateway and downstream of market data, and are outside this codebase.

The wire schema is `sbe/src/main/resources/message-schema.xml`; codecs are generated from it and byte offsets are never written by hand.

> **SBE**
> Simple Binary Encoding — a fixed-layout binary message format with generated codecs. Messages are encoded directly into the Aeron log buffer through `tryClaim`, so publishing a message copies nothing.

1.4 The two kinds of configuration

Everything an operator sets falls into exactly one of two categories, and confusing them is the most common source of a shard that starts but is subtly wrong.

Geometry and topology — which securities exist, on which shard, with what tick size, price floor, ladder size and order capacity; which gateway speaks for which participant; where the gateway's client endpoints are. This is

authored in the control plane, published as an immutable numbered **release**, and read from a *file* at boot by every process that needs it. Every process prints a `fingerprint()` of what it read at startup, and those fingerprints must agree.

Node-local settings — Aeron directories, cluster directories, feed channels, metrics switches, thread and idle settings. These live in each process's own properties file and are templated per host. They are deliberately not in the database and not in a release.

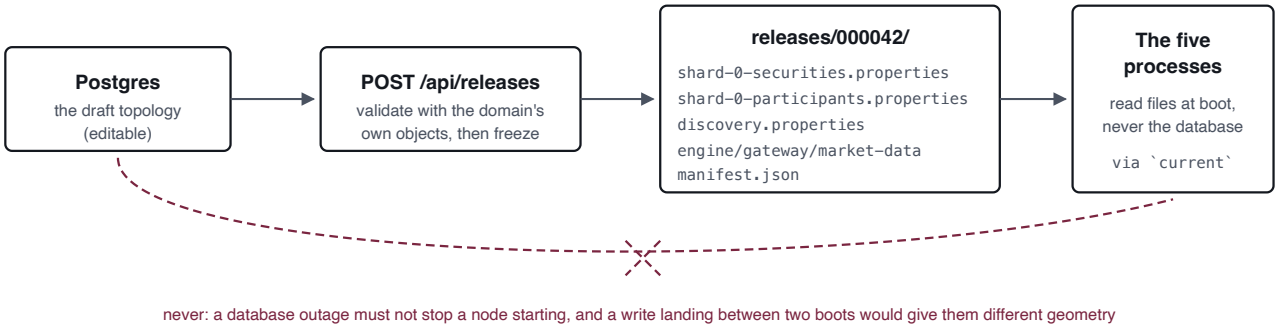


Figure 1.2 A release is authored in Postgres, frozen into files, and read from files. The database is never on a boot path.

FINGERPRINT

A short hash over a shard's published geometry, printed by the engine, gateway, market-data and discovery processes at startup. Three nodes printing three different values is a divergence waiting to happen on the first order, and consensus cannot catch it — this is the one misconfiguration Raft will not save you from. The engine catches it instead. The leader writes its fingerprint into the log at each term start, and a node that disagrees refuses to go on.

RELEASE

An immutable, numbered directory of rendered configuration artifacts produced by `POST /api/releases`. Republishing never overwrites; it allocates the next version. Deployments point a `current` symlink at a release directory, and processes read through that symlink.

1.5 Prices, quantities and identifiers

Prices and quantities are fixed-point signed 64-bit integers with **eight implied decimals**. There is no floating point anywhere in pricing or matching arithmetic, and no operator-facing interface should introduce one.

Displayed	On the wire
`100.00`	`10000000000`
`0.01` (a tick)	`1000000`
`101.30`	`10130000000`

`securityId` is a 32-bit integer and is the primary identity of an instrument everywhere on the wire; symbols exist for humans and for routing lookups. `participantId` identifies the trading firm. `exchangeOrderId` is allocated by the engine and is what a cancel refers to.

1.6 Session phases

Every security is in one of four phases. Orders are **accepted and booked in every phase except** `CLOSED`; only *matching* is gated on `CONTINUOUS`. A crossed book outside continuous trading is therefore normal, and is resolved by the uncrossing algorithm.

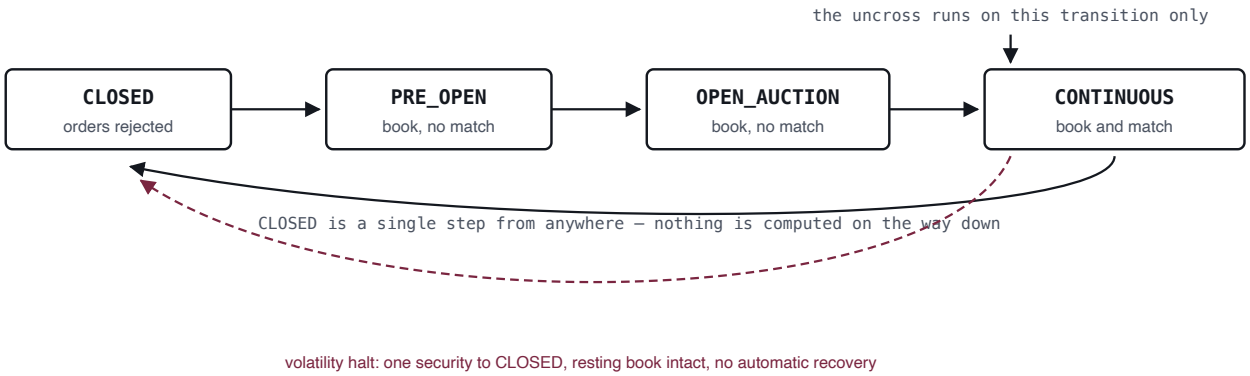


Figure 1.3 The session lifecycle. Phases are walked, not jumped: the uncross runs on exactly one transition.

The transition that matters is `OPEN_AUCTION → CONTINUOUS`, which runs the uncross. Sending `CONTINUOUS` from any other phase is accepted and silently skips the auction, leaving a crossed resting book crossed until an aggressor happens to arrive.

A **session transition is shard-wide**. There is no per-security session command, so moving one security's phase moves every book on that shard.

1.7 What is not built

The manual marks unimplemented behaviour inline, in *todo* callouts, at the point where an operator would otherwise expect it to work. The consolidated list is in 6.9.

1.8 How to read this manual

Section 2 brings the whole system up on one machine and is the fastest way to acquire the vocabulary the rest of the manual uses. Section 3 is the production topology. Section 4 is the complete configuration reference and is meant to be consulted rather than read. Section 5 is the daily operational procedure — order flow, market data, the trading day, and what to measure. Section 6 is diagnosis and maintenance. Section 7 indexes the terms and commands by section.

Command transcripts in this manual are real output from a running system, not illustrations.

Companion documents in the repository, which this manual summarises rather than replaces: `docs/Design.md` is the authoritative specification, `docs/ControlPlane.md` covers the control plane in depth, `docs/ProdDeployment.md` the production tier, `docs/LocalTesting.md` the manual walkthrough, and `deploy/README.md` the development stack.

2. Development deployment and operation

This section brings up the complete system — control plane, database, admin UI and one full shard — on a single machine, and drives a trade through it. Everything in later sections assumes the vocabulary established here.

> **A DEVELOPMENT STACK IS NOT A SMALL PRODUCTION STACK**
>
> The stack in `deploy/` runs **one** cluster node, a shared secret committed to the repository, a fixed admin password, and dynamic MDC in place of multicast. Each of those is listed in 2.9 with what to do instead. Do not carry them forward.

2.1 Prerequisites

JDK	21 or later
Docker	with Compose v2 (`docker compose`, not `docker-compose`)
Node	20+, only for the admin UI and for building this manual
Memory	~6 GB free; the two Aeron tmpfs volumes are sized 1 GB each

The JVM needs two module opens for Agrona's buffer intrinsics and the Aeron driver. They are already set on every Gradle test and run task, and in every start script the build produces:

```
--add-opens java.base/jdk.internal.misc=ALL-UNNAMED
--add-opens java.base/sun.nio.ch=ALL-UNNAMED
```

Without them the first `UnsafeBuffer` allocation throws `IllegalAccessError`.

2.2 Build

```
./gradlew build          # compile, generate SBE codecs, run the test suite
./gradlew installDist    # start scripts under */build/install, which the images copy
```

`./gradlew build` is npm-free by design; the Vue frontend in `web/` is a normal frontend project and is built separately.

> **WARNINGS ARE ERRORS**
>
> Every module compiles with warnings as errors, `control` included. Do not disable it to get a build through — a silently un-inlined function would break the engine's zero-allocation profile without failing anything.

2.3 The whole stack at once

```
./gradlew installDist
cd deploy
docker compose up -d           # roughly 40 seconds to a trading market
```

Admin UI	http://localhost:8081 — `admin` / `most-dev-password`
Control API	http://localhost:8080
Teardown	`docker compose down` — add `-v` to discard the books as well

`docker compose up` also runs a one-shot `seed` container that authors, over the REST API, what the processes booted from: shard 0 and its securities (imported from the very file they read), participants 7, 8 and 20–23, the gateway `gw-0` and the control plane's own operator identity `control` with the secrets the processes present, and the `equities` calendar. It publishes a release, seeds reference prices and walks the session to `CONTINUOUS`, so `deploy/init/seed.sh` doubles as a worked example of driving the exchange the supported way.

```
seed: creating shard 0
seed: importing the shard security file the processes booted from
seed: import said: {"shardId":0,"fingerprint":"3e04cf2b9902f08a","inserted":
["AAPL","MSFT"],"updated":[]}
seed: participant 7: 201
...
seed: gateway gw-0 secret: 200
seed: gateway control secret: 200
seed: publishing release
seed: release registry fingerprint for shard 0: 4c9c24c10df3ff56
seed: creating the equities schedule
seed: seeding definition for security 1
seed: seeding definition for security 2
seed: session → PRE_OPEN
seed: session → OPEN_AUCTION
seed: session → CONTINUOUS
seed: done -- shard 0 is CONTINUOUS with AAPL and MSFT at 100.00
```

REBUILDING THE CLI IMAGE RESTARTS THE SHARD

`most` and `cluster-host` run the same tools image, and `docker compose up --build` does not rebuild it — the `most` service sits in the `cli` profile, so it needs `docker compose --profile cli build`. Once rebuilt, the next `docker compose up` **or** `docker compose run` recreates the cluster host, and a cluster host recreated within about ten seconds of the old one stopping refuses to start (`active mark file detected`). Stop the shard, wait out the window, then start it; use `docker compose run --no-deps` for one-off containers.

What each container is

Service	What it is
`cluster-host`	The shard's media driver, archive and consensus module. Holds the `shard0` network alias.
`engine`	The service container only. Attaches to `cluster-host` over shared memory.
`gateway`	Order entry, with its cluster identity `gw-0`, which it also enforces (4.3).
`market-data`	L1/L2/L3 and the snapshot feed.
`discovery`	The directory broadcast.
`control`, `postgres`, `web`	The control plane, its database, and the SPA behind nginx.
`control-driver`	The control plane's *own* media driver, so it reaches the shard over the network the way an adapter will rather than by mounting the engine's shared memory.
`most`	A run-only container for the CLI. Started on demand by `deploy/most`.

MEDIA DRIVER

The Aeron process that owns the shared-memory buffers and the sockets. Every process on a node talks to one through a directory of memory-mapped files. It allocates, which is why it is never inside the engine binary.

2.4 Verifying the stack came up

Every process prints its geometry fingerprint at startup. They must all agree.

```
$ docker compose logs cluster-host engine gateway market-data discovery | grep -i fingerprint

cluster:         participants=/config/shard-0-participants.properties fingerprint=4c9c24c10df3ff56
gateways=[control, gw-0] reload=5000ms
matching-engine: shard=0 fingerprint=3e04cf2b9902f08a securities=[AAPL, MSFT] serviceId=0
clusterDir=/cluster/cluster participantRegistry=4c9c24c10df3ff56 reload=5000ms
gateway:         shard=0 fingerprint=3e04cf2b9902f08a securities=[AAPL, MSFT] in=aeron:udp?
endpoint=0.0.0.0:20001:20 out=...
gateway:         identity=gw-0 registry=4c9c24c10df3ff56 reload=5000ms participants=[7, 8, 20, 21,
22, 23] cancelOnly=[] operator=true
market-data:     shard=0 fingerprint=3e04cf2b9902f08a securities=[AAPL, MSFT] in=aeron:ipc:12 ...
```

Two distinct fingerprints appear here and both are correct: `3e04cf2b9902f08a` is the **shard's geometry**, and `4c9c24c10df3ff56` is the **participant registry's**. They are deliberately separate values — rotating a gateway's secret is not a change of geometry, and must not invalidate a release. The `seed` container prints the registry fingerprint of the release it published (`seed: release registry fingerprint for shard 0: 4c9c24c10df3ff56`), which is the check that the control plane's database and the file the shard booted from say the same thing. The gateway's `identity=` line is what it will **enforce** (4.3): `gw-0` speaks for 7, 8 and the load generator's 20–23, and is the operator gateway the CLI's and the control plane's commands pass through.

The five processes should each print a readiness line:

```
cluster:         started, awaiting shutdown signal
matching-engine: started, awaiting shutdown signal
gateway:         started
market-data:     started
discovery:       started
```

2.5 The operator CLI

`most` is the operator command-line tool. Against the dev stack, use the `deploy/most` wrapper — it runs the CLI in a container with its own media driver, on the outside of the shard, exactly where a FIX adapter will sit.

Every subcommand begins by waiting for a **discovery broadcast**, because the routing table is the only thing that maps a symbol to the shard and gateway serving it. A `most` command that appears to hang is almost always waiting for that.

```
$ ./most securities --verbose

SYMBOL     ISIN           SHARD CCY    NAME
AAPL       US0378331005   0     USD    Apple Inc.
           id=1 tick=0.01 floor=0.00 levels=32768
           orders → aeron:udp?endpoint=shard0:20001:20   reports ←
aeron:udp?...control=shard0:20002...:21
MSFT       US5949181045   0     USD    Microsoft Corporation
           id=2 tick=0.01 floor=0.00 levels=32768
           orders → aeron:udp?endpoint=shard0:20001:20   reports ←
aeron:udp?...control=shard0:20002...:21

2 securities, universe version 9181280125937456696
```

The **universe version** is a 64-bit hash of the whole published directory. It is how a subscriber detects that routing has changed underneath it.

2.6 Driving a trade

Rest an order, then cross it from a second participant. The dev stack's book is rarely empty — a load run leaves thousands of orders resting — so these orders are priced inside the spread, where they meet only each other.

```
$ ./most send --symbol MSFT --side sell --price 100.01 --qty 10 --clordid ORD-2001 --participant 7
--follow 2

sending sell 10 MSFT @ 100.01 clOrdId=ORD-2001
sent to shard 0 via aeron:udp?endpoint=shard0:20001
  NEW  MSFT  orderId=20032 clOrdId=ORD-2001  leaves 10 cum 0 of 10  seq 32

$ ./most send --symbol MSFT --side buy --price 100.01 --qty 4 --clordid ORD-2002 --participant 8 --
follow 2

sending buy 4 MSFT @ 100.01 clOrdId=ORD-2002
sent to shard 0 via aeron:udp?endpoint=shard0:20001
  NEW  MSFT  orderId=20033 clOrdId=ORD-2002  leaves 4 cum 0 of 4  seq 24
  TRADE  MSFT  orderId=20033 clOrdId=ORD-2002  last 4 @ 100.01 cum 4 leaves 0  seq 25
```

`clOrdId` is the participant's own id, returned on every report exactly as it was sent (§5.4). `orderId` is the `exchangeOrderId` the engine assigned, and `seq` the participant's report sequence: each participant numbers separately, which is why participant 8's reports are at 24 while 7's are at 32. Without `--clordid` the CLI uses the current time in milliseconds.

The maker's own `TRADE` report went to participant 7 after its `send` had stopped listening, and took sequence 33. Cancelling the partially filled maker shows the next one, and both quantities the engine states on every report:

```
$ ./most cancel --symbol MSFT --side sell --order-id 20032 --orig-clordid ORD-2001 --clordid CXL-
2001 --participant 7 --follow 2

cancelling orderId=20032 on MSFT
sent to shard 0 via aeron:udp?endpoint=shard0:20001
  CANCELED  MSFT  orderId=20032 clOrdId=CXL-2001 origClOrdId=ORD-2001  leaves 0 cum 4 of 10  seq 34
```

A cancel names the order by `--order-id` and `--orig-clordid` together, and the engine cancels only when both match an order of the same participant. Its report carries the cancel's own `clOrdId` and the order's as `origClOrdId`; the CLI prints `origClOrdId` only when the two differ.

> **CUMQTY AND ORIGQTY**
>
> `cumQty` is the quantity filled so far and `origQty` the quantity originally submitted; both are **stated by the engine on every execution report** and are never derived by a client. A terminal report carries `leavesQty = 0` whether the order filled or was cancelled, so subtracting `origQty - leavesQty` would report this cancelled order as fully filled. An `origQty` of `0` means *unknown*, which reaches a client as an explicit `UNKNOWN` enrichment rather than as a number.

A rejection is an execution report like any other, and takes a sequence number like any other:

```
$ ./most send --symbol AAPL --side buy --price 500.00 --qty 1 --clordid ORD-3001 --participant 7 --
follow 2

sending buy 1 AAPL @ 500.00 clOrdId=ORD-3001
sent to shard 0 via aeron:udp?endpoint=shard0:20001
  REJECTED  AAPL  orderId=0 clOrdId=ORD-3001  reason PRICE_OUT_OF_BOUNDS  seq 35
```

Two mistakes never reach the wire. The CLI resolves a symbol against the directory first, and checks a client order id against the same rule the gateway enforces:

```
$ ./most send --symbol TSLA --side buy --price 100.00 --qty 1 --clordid ORD-3002
most: unknown symbol 'TSLA' -- try `most securities`

$ ./most send --symbol AAPL --side buy --price 99.00 --qty 1 --clordid ORD#3003 --participant 7
most: not a valid clOrdId (1-20 of A-Z a-z 0-9 -_+;:./): "ORD#3003"
```

2.7 Watching the book

`most book` rebuilds order books from the L2 depth feed and its periodic snapshot, using the same assembler any other consumer would use.

```
$ ./most book --symbol AAPL,MSFT --depth 5 --refresh 500

── AAPL (id 1) ──────────────────────────────────
       qty ords          bid │ ask        qty        ords
        89 (1)         99.98 │
        79 (1)         99.97 │
        65 (1)         99.95 │
        81 (1)         99.94 │
        28 (1)         99.89 │
  spread n/a   updates 387

── MSFT (id 2) ──────────────────────────────────
       qty ords          bid │ ask        qty        ords
        54 (1)         99.96 │ 100.02     98         (1)
       148 (2)         99.91 │ 100.04     17         (1)
        94 (1)         99.87 │ 100.12     31         (1)
       130 (3)         99.86 │ 100.14     29         (1)
       335 (9)         99.85 │ 100.17     74         (1)
  spread 0.06   last 100.01 x 4   updates 760
```

If it prints `waiting for the depth feed...` it has not yet received a snapshot for that security. The snapshot feed cycles one security per interval, so a joiner waits at most one full pass.

> **SYNCHRONISED**
>
> A depth subscriber is *synchronised* once it has installed a snapshot image and can apply increments to it. Before that it publishes **no depth at all** rather than a partial ladder: “waiting for a snapshot” and “no liquidity” are different answers and must never render the same way.

2.8 The admin UI

The console is at http://localhost:8081. It is a Vue SPA over the control plane’s REST API, served same-origin — the session and CSRF cookies are same-origin mechanisms, so the SPA and the API sit behind one host.

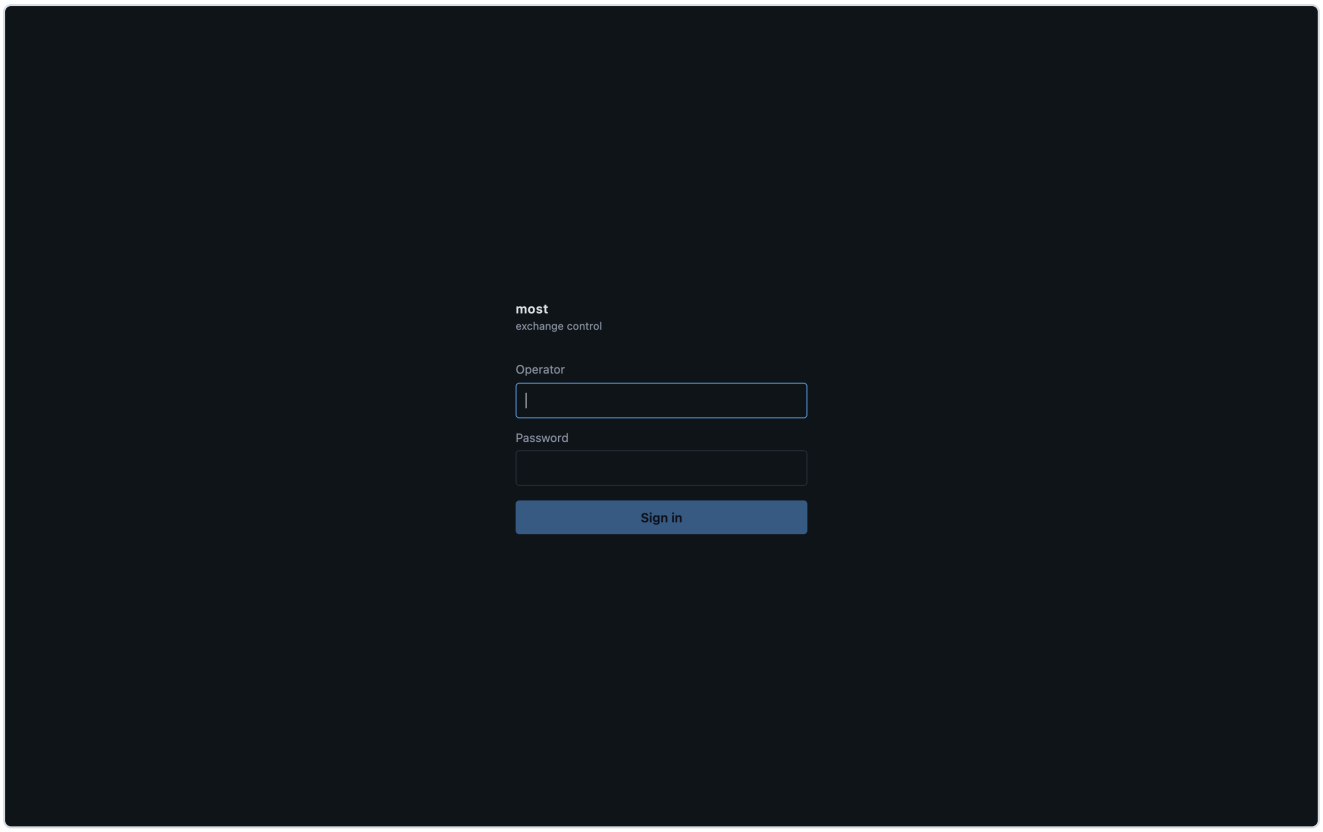


Figure 2.1 The sign-in screen. Every route but this one requires an authenticated operator; the router guard exists so an operator sees a form rather than empty tables, and the server's 401 is what actually protects the data.

Status is the first screen and answers a question no other screen does: what the exchange is actually doing, as opposed to what the database says it should be. It is derived from the L3 book event feed.

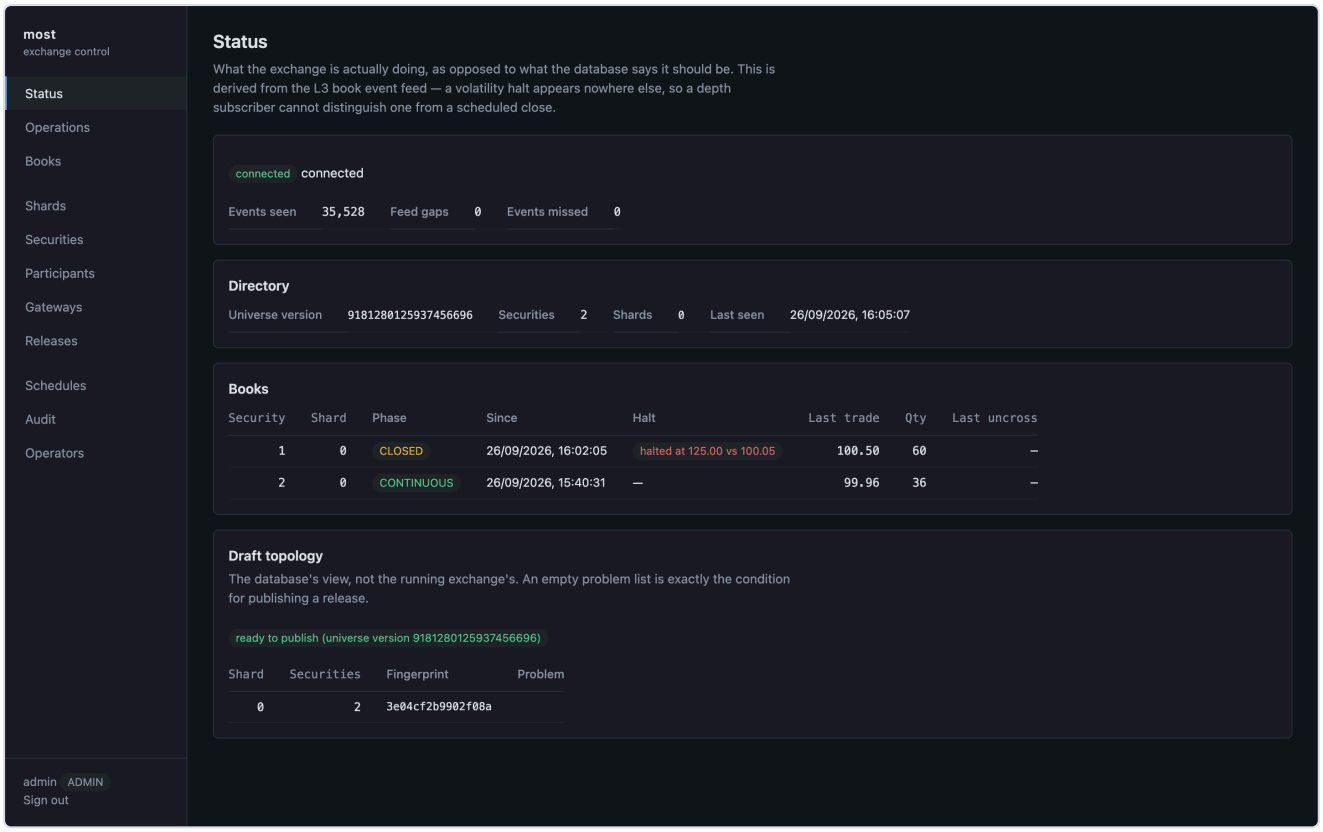


Figure 2.2 Status. Observed phase per book from the L3 feed, the directory the shard is broadcasting, and the draft topology's readiness to publish.

Books renders live depth, streamed over server-sent events, conflated to four images a second. A book that is not synchronised draws no ladder at all.

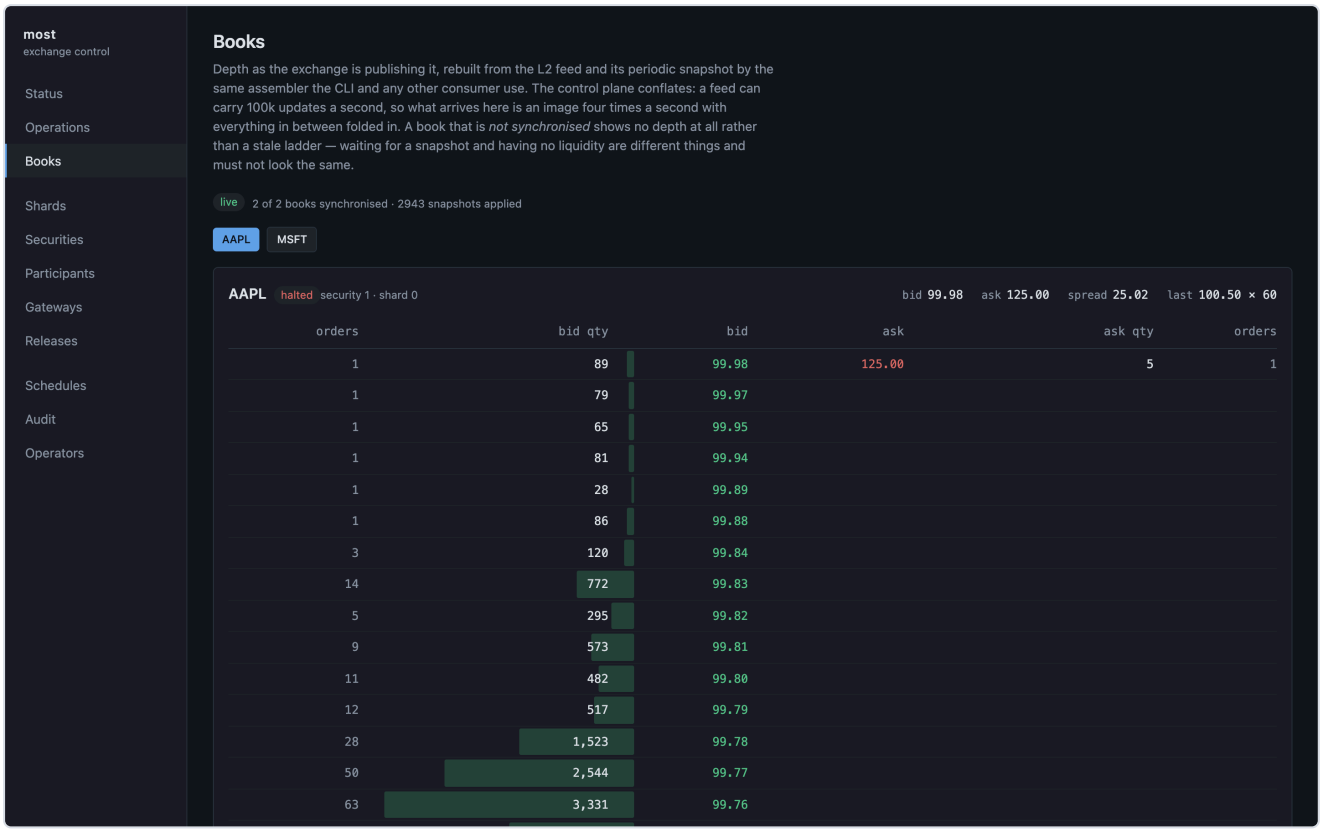


Figure 2.3 Books. Depth rebuilt by the same assembler the CLI uses, conflated on the feed thread so a browser is never on the market data critical path.

Operations is the only screen that touches a running market.

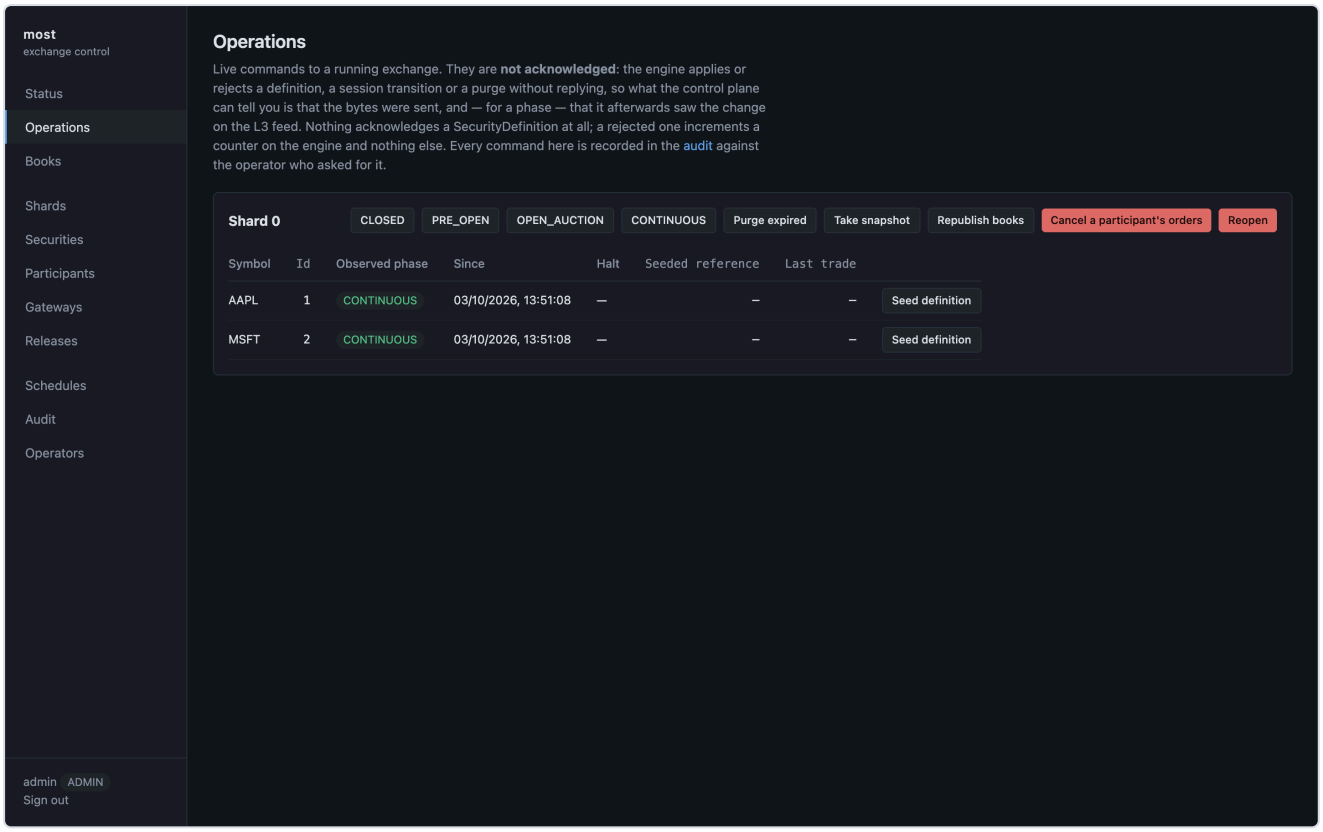


Figure 2.4 Operations. Session transitions, definition seeding, the expiry purge, a snapshot request, a book image request, the bulk cancel of one participant's orders, and halt recovery.

The remaining screens are covered where they are used: Shards, Securities, Participants, Gateways and Releases in section 4; Schedules, Audit and Operators in section 5.

2.9 What is development-only

Development stack	Production
One cluster node	Three or five members on separate machines (3.2)
`config/gateway-0.secret` and `config/control.secret` committed to the repository	Secrets issued by the control plane, delivered out of band (4.3, 4.10)
A fixed, published admin password	`CONTROL_ADMIN_PASSWORD` unset, so one is generated and logged once (4.9)
`CONTROL_COOKIE_SECURE=false`	`true`, behind TLS
Dynamic MDC for market data and the gateway's outbound leg	UDP multicast with an IGMP querier (3.7)
The scheduler disabled	Enabled — it is what bounds restart time by snapshotting at each close (5.6)
Hand-written `shard-0-securities.properties`	A published release, pointed at by a `current` symlink (4.2)
`ThreadingMode.SHARED`, no core isolation	Dedicated threading and isolated cores (3.4)

Development stack	Production
The gateway on the shard's own media driver, so IPC egress (`gateway.egressChannel=aeron:ipc`) is available — the highest-throughput setting (4.8)	A separate gateway tier on UDP egress (3.2), until a gateway can follow the leader
Engine, gateway and market data set to `backoff` (`*.idleStrategy`)	`busyspin`, the default, on isolated cores (3.4)
Duty-cycle counters on (`*.metrics`, `most cluster --duty`)	Optional; cheap enough to leave on (5.8)

> **ONE MEDIA DRIVER IS ONE NETWORK IDENTITY**
>
> In the development stack the five shard processes share a media driver that runs in the `cluster-host` container, so **every UDP endpoint the shard exposes is bound there** and must name `shard0`, never the individual process's container name. A channel naming `gateway:20002` fails with `Cannot assign requested address`. This is not a Docker workaround; it is what "these five processes are one node sharing `/dev/shm`" means once the processes are in separate containers.

2.10 Running without Docker

Useful when iterating on one process. Five terminals, started in this order — the engine attaches to a consensus module that must already exist.

```
# 1. cluster host: media driver + archive + consensus module
./tools/build/install/tools/bin/most cluster --dir build/cluster-host \
    --participants config/shard-0-participants.properties
# wait for: cluster: started, awaiting shutdown signal

# 2. engine — the cluster service container only
./engine/build/install/engine/bin/engine config/engine.properties
# wait for: matching-engine: started, awaiting shutdown signal

# 3. gateway
./gateway/build/install/gateway/bin/gateway config/gateway.properties

# 4. market data
./market-data/build/install/market-data/bin/market-data config/market-data.properties

# 5. discovery
./discovery/build/install/discovery/bin/discovery config/discovery.properties
```

The control plane needs Postgres and one environment variable:

```
docker run -d --name most-control-db -p 5432:5432 \
  -e POSTGRES_DB=most_control -e POSTGRES_USER=most -e POSTGRES_PASSWORD=most postgres:16-alpine

./gradlew :control:installDist
CONTROL_ADMIN_PASSWORD=... ./control/build/install/control/bin/control
```

And the frontend, which proxies `/api` to the control plane so both are one origin:

```
cd web && npm install && npm run dev     # http://localhost:5173
```

2.11 Automated end-to-end checks

These scripts exercise things unit tests cannot. Run the first two after changing anything that touches the wire or process lifecycle.

Script	What it proves
`e2e/run-e2e.sh`	Every process against a real single-node cluster: a trade driven through the CLI, then a short `most load` to confirm the harness still correlates reports to orders. Takes each binary path from an environment variable (`ENGINE`, `GATEWAY`, `MARKETDATA`, `DISCOVERY`) so the same run can drive native images.
`e2e/run-restart.sh`	That state survives a restart. Rests orders, snapshots, stops the whole node, and restarts it on the same security file; on one with a security removed while it holds orders (must refuse); on one with an emptied security removed (must start); and on one with a security added. Also places an order through one gateway and cancels it through another.
`e2e/run-upgrade.sh`	That a version 7 shard upgrades to schema version 8: orders rest on version 7, `most cluster shutdown`, then version 8 restores them, a cancel names an old integer id as its decimal string, `most cluster shutdown` on version 8 exits both processes by itself, and a version 7 log tail with no snapshot at its end is refused. Takes a version 7 build as `OLD`.
`e2e/run-attribution.sh`	Splits a client round trip into gateway, engine and everything else, writing `.hgrm` histograms that are meant to be diffed across changes.
`e2e/run-epsilon-soak.sh`	Runs the Epsilon-GC binary against a real cluster and measures the allocation slope across two runs.

`run-e2e.sh` wipes everything and starts fresh, so it says nothing about whether state survives — that is `run-restart.sh` and only `run-restart.sh`.

2.12 Shutting down

```
cd deploy
docker compose exec -T cluster-host most cluster shutdown --dir /cluster
                           # snapshot, then cluster-host and engine exit 0 by themselves
docker compose down        # keeps the books: cluster-data is a durable named volume
docker compose down -v     # discards the archive, the cluster directory and the books
```

The snapshot is what makes the next start quick. Without one, `docker compose down` still keeps the books, but the next start replays the log from genesis to rebuild them (5.6).

> **A NODE CANNOT RESTART IMMEDIATELY AFTER THE PREVIOUS ONE STOPPED**
>
> Aeron's archive and cluster **mark files** carry a liveness timestamp, and a new process refuses to start until it ages out — roughly ten seconds, even after a clean shutdown. An automatic restart policy must wait longer than that or it will fail on `active mark file detected` and back off.

3. Production deployment

3.1 The rule that shapes everything

A shard is not orchestrated. The three cluster nodes are long-lived machines with stable hostnames, isolated cores, huge pages and a dedicated NVMe device for the archive. They are not pods, they are not autoscaled, and they are not rescheduled. Everything with no realtime requirement — the control plane, its database, the admin UI, protocol adapters — goes to an ordinary orchestrated tier.

That line is not a preference. The engine is a single thread whose working set is a cache-line packed order pool; a rescheduled workload, a shared core or a cross-socket memory access costs more than everything the matching logic does.

> **NOTHING IN THIS SECTION HAS BEEN RUN END TO END**
> The port map, member string and unit files below are derived from the code and from the development stack, not from a working production deployment. 3.10 lists the specific code changes that are blocking, and every one of them is small.

3.2 Topology

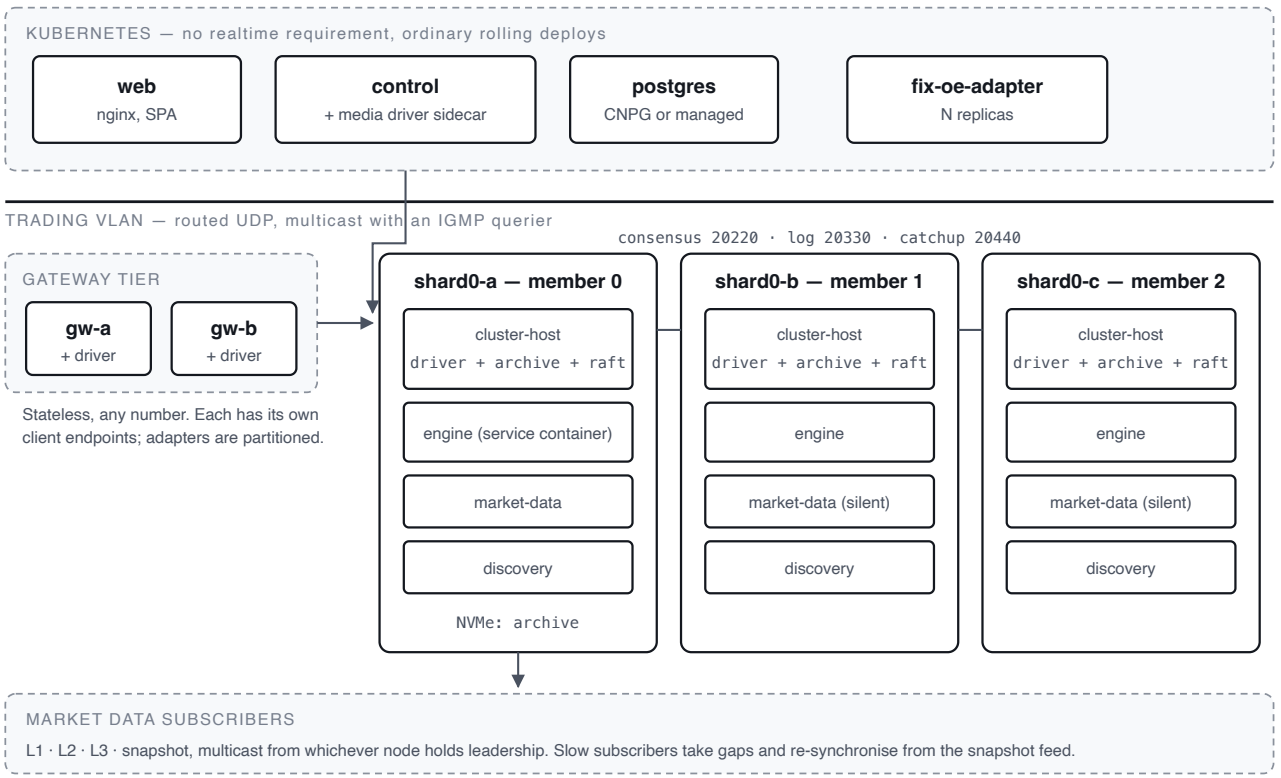


Figure 3.1 The production topology: three cluster nodes on bare metal, a separate stateless gateway tier, and everything without a realtime requirement in Kubernetes.

What runs where

Every node runs `cluster-host` and `engine`. That is what a Raft group is.

Every node runs `market-data`, and only one of them emits. This falls out of the design rather than needing coordination: the book event stream is `aeron:ipc` and is leader-only, muted explicitly on role change because it is a plain publication and not cluster egress. A follower's market-data process is subscribed to a silent stream and publishes nothing. On failover the new leader's engine starts publishing and its co-located market-data picks up. There is no standby logic and no cross-machine hop for a stream that was deliberately made shared-memory.

Every node may run `discovery`. It is a stateless repeating broadcast and a duplicate cycle is harmless — each cycle is staged and replaces the routing table wholesale, so a truncated cycle never replaces a good table.

Where the gateway runs is a choice: `gateway.placement`. A gateway holds no per-order state, so either placement is correct. They differ in throughput and in what a failover costs a client (4.5, 4.8).

- `independent` (the default): gateways are their own tier, on their own machines, as drawn above. An `AeronCluster` client lists every member and follows the leader, so a failover is invisible to adapters except for how long it takes. Execution reports come back over UDP, which caps one node at about 500,000 orders/s with the default datagram size (4.8). Larger datagrams raise it, and on a real network they need jumbo frames on the trading VLAN.
- `colocated`: a gateway on **each** core machine, using that machine's media driver, IPC both ways. About three times the throughput (4.8). Only the leader's gateway is active. The others refuse every order with `GATEWAY_UNAVAILABLE`, so adapters hold all three machines' endpoints and move to the next on that reject. They also move when their publication disconnects, because when the leader's *machine* dies its gateway dies with it and sends nothing. All three gateways present **the same** registry identity, so the one that becomes active takes every participant's routes the moment it connects. The cluster hosts run `most cluster --ipc-ingress`.

Pick one per shard. `e2e/run-failover.sh` runs either through two failovers and prints what each cost.

> **GATEWAYS MUST HAVE DISJOINT CLIENT ENDPOINTS**
>
> `gateway.client.inbound.channel` is a single address. Two gateways subscribed to **one** inbound channel each receive every order and forward both — that is duplicate orders, not redundancy. Give each gateway its own client endpoints and partition adapters across them. Losing one costs its adapters a reconnect and nothing else; a peer reports correctly on orders it never saw.

> **A FAILOVER COSTS A CO-LOCATED CLIENT ITS IN-FLIGHT ORDERS, AND A BLIND WINDOW**
>
> When the leader's machine dies, a client keeps publishing to its dead gateway until Aeron's publication connection timeout (5 s by default, on the client's driver) declares it gone. Every order sent in that window is lost without a reply. Measured on one machine at 2,000 orders/s: ~5 s and 9,978 orders, against ~3.5 s and 6,412 for an independent gateway, whose window is the election itself. In both placements nothing tells a client what became of an unanswered order; reconcile with a cancel-all (5.3) and re-enter.

> **⚠ THE DIRECTORY ADVERTISES ONE GATEWAY PER SHARD**
>
> `ShardEntry` carries a single order-entry channel and `DirectoryClient` keeps one route per shard, so several gateways for one shard cannot all be advertised. A virtual address in front of the gateway tier is the interim answer; naming them individually is a wire change.

3.3 Machine preparation

Hardware

	Recommendation	Why
CPU	16+ physical cores, x86-64-v3 or better, **one socket**	~12 threads need cores; one socket avoids NUMA on the hot path
RAM	64 GB minimum	~1.5 GB per shard of pools, the client order ids and the report ring, plus Aeron log buffers, plus page cache for the archive
Storage	NVMe SSD dedicated to the archive	The archive writes every replicated message and is the single largest contributor to round-trip latency
Network	Dual 10 GbE, one dedicated to the trading VLAN	Multicast market data and consensus must not share a link with management traffic

Single socket matters more than it looks. A cross-socket memory access on the matching path costs more than the matching itself.

Kernel and boot

```
# /etc/default/grub — GRUB_CMDLINE_LINUX
isolcpus=2-13 nohz_full=2-13 rcu_nocbs=2-13
hugepagesz=1G hugepages=8 default_hugepagesz=1G
intel_idle.max_cstate=0 processor.max_cstate=1 idle=poll
mce=ignore_ce
```

> **HUGE PAGES ARE MANDATORY, NOT OPTIONAL**
> A 0.9 GB order pool on 4 KB pages is roughly 230,000 TLB entries, for a structure that was cache-line packed specifically to cost one cache miss per order. The TLB miss undoes the entire design.

`idle=poll` and the C-state pinning trade power for the absence of wake-up latency. On a machine whose only job is one exchange that is the right trade — and it is also why these machines should run nothing else.

```
tuned-adm profile latency-performance

# NIC: no interrupt coalescing on the trading interface, IRQs pinned to housekeeping cores
ethtool -C ens1f0 rx-usecs 0 rx-frames 1
ethtool -G ens1f0 rx 4096 tx 4096
```

3.4 Core allocation

Cores 0–1 are housekeeping — kernel, IRQs, ssh, metrics agents. Cores 2–13 are isolated.

Cores	Thread
2, 3, 4	Media driver conductor, sender, receiver (`ThreadingMode.DEDICATED`)
5, 6	Archive conductor, recorder (`ArchiveThreadingMode.DEDICATED`)
7	Consensus module
8	**Engine service container — the matching thread**

Cores	Thread
9, 10	Gateway and its client conductor, **only if** a gateway is co-located on the node for IPC egress (3.2, 4.8); spare in the separate-tier topology
11, 12	Market-data, and its client conductor
13	Discovery

Core 8 is the one that matters. Everything else is plumbing around it.

> **IDLE STRATEGIES ARE NOT PORTABLE BETWEEN MACHINES**
> With dedicated cores, `BusySpinIdleStrategy` is correct for the driver threads, the consensus module and the service container. **Without** isolated cores it is actively harmful — spinning threads starve the kernel's own softirq processing. Never copy production idle strategies onto a development machine.

The two driver-side modes in that table are the ones you must ask for. `most cluster` defaults to `SHARED` for both, which is right for a laptop and wrong here:

```
most cluster --members "$MEMBERS" --host shard0-a \
     --driver-threading DEDICATED --archive-threading DEDICATED \
     --participants /etc/most/current/participants-shard0.properties
```

`--driver-threading DEDICATED` **is worth 1.6x of throughput** and 81x of median latency at the edge (4.8). The next ceiling after it is the gateway's egress channel (4.8, "The egress channel"). `--archive-threading DEDICATED` earns its place only *with* it and only with the cores above isolated — on shared cores it is measurably worse than leaving it alone.

Idle strategies are configuration. The engine, gateway and market-data each take one — `engine.idleStrategy`, `gateway.idleStrategy`, `md.idleStrategy` (4.4–4.6) — and **all three default to** `busyspin`, which is right for the layout above and needs nothing set. The cluster host's threads use Aeron's own settings, which back off by default; on isolated cores, make them spin with Aeron's system properties in the cluster host's `JAVA_OPTS`:

```
JAVA_OPTS="-Daeron.conductor.idle.strategy=org.agrona.concurrent.BusySpinIdleStrategy \
           -Daeron.sender.idle.strategy=org.agrona.concurrent.BusySpinIdleStrategy \
           -Daeron.receiver.idle.strategy=org.agrona.concurrent.BusySpinIdleStrategy \
           -Daeron.archive.idle.strategy=org.agrona.concurrent.BusySpinIdleStrategy \
           -Daeron.archive.recorder.idle.strategy=org.agrona.concurrent.BusySpinIdleStrategy \
           -Daeron.cluster.idle.strategy=org.agrona.concurrent.BusySpinIdleStrategy"
```

On a machine **without** a core per thread, set `backoff` on anything that is mostly idle. A spinning loop at 13% of its real work still takes 100% of a core, and on a development laptop that core was the difference between keeping up and not (5.8).

> **THIS ALSO MEANS CPU% TELLS YOU VERY LITTLE ABOUT THESE PROCESSES**
> Because the engine, gateway and market-data busy-spin, each reads ~100% of a core whether it is working or idling. `top` cannot tell you whether one of them is the constraint. **Their duty-cycle counters can** — the share of its core each thread actually spends working, live (5.8).

3.5 Aeron buffer sizing

Aeron's defaults are sized for throughput, not for a fixed memory budget. On 64 GB the UDP defaults are fine; the IPC default deserves attention, because a 64 MB IPC term means roughly a 192 MB log buffer for the book event

stream alone.

```
aeron.term.buffer.length=16m
aeron.ipc.term.buffer.length=16m
aeron.socket.so_sndbuf=2m
aeron.socket.so_rcvbuf=2m
aeron.rcv.initial.window.length=2m
```

> **A DRIVER THAT CANNOT MAP A BUFFER DOES NOT FAIL POLITELY**
> It takes SIGBUS on the next page touch and reports `InternalError: a fault occurred in an unsafe memory access operation`, which reads like a JVM bug rather than a sizing problem. Size the shared-memory mount for the number of subscriptions the process will hold: every subscription image is a log buffer of three terms.

3.6 Cluster configuration

Member string

Aeron's format is `memberId,ingress,consensus,log,catchup,archiveControl`, members separated by `|`. The ports are the CLI's existing defaults and are fine to keep — the three members are on different machines, so they do not need to differ.

```
0,shard0-a:20110,shard0-a:20220,shard0-a:20330,shard0-a:20440,shard0-a:8010|\
1,shard0-b:20110,shard0-b:20220,shard0-b:20330,shard0-b:20440,shard0-b:8010|\
2,shard0-c:20110,shard0-c:20220,shard0-c:20330,shard0-c:20440,shard0-c:8010
```

Pass it identically to all three with `most cluster --members`, and give each its own `--member-id 0`, `1` or `2`. A node's archive listens on its own entry's archive endpoint. `e2e/run-cluster3.sh` runs three members on one machine through an election, two failovers and a member rejoining from its snapshot.

Port map

Every one of these is bound by the **media driver** on its own machine.

Port	Bound by	Purpose
20110	consensus	Cluster ingress — gateways and the control plane connect here
20220	consensus	Consensus, member to member
20330	consensus	Log replication
20440	consensus	Catchup
8010	archive	Archive control
20001	gateway	**Client inbound** — adapters publish orders here
20002	gateway	**Client outbound** control address — execution reports
40000	discovery	Directory broadcast
40001 / 40002 / 40003	market-data	L1 / L2 / L3
40004	market-data	L2 recovery snapshot feed

20110, 20001 and 20002 are the only ports anything outside the trading VLAN needs to reach. Firewall the rest to the three machines and the market-data subscriber group.

Gateway ingress

The gateway lists every member so its cluster client can find the leader:

```
gateway.ingressChannel=aeron:udp
gateway.ingressEndpoints=0=shard0-a:20110,1=shard0-b:20110,2=shard0-c:20110
```

3.7 Market data: multicast

The development stack uses dynamic MDC because a Docker bridge does not route multicast. On a real VLAN, use what the design specifies:

```
md.l1.channel=aeron:udp?endpoint=239.10.0.1:40001|interface=10.20.0.0/24
md.l1.streamId=1
md.l2.channel=aeron:udp?endpoint=239.10.0.2:40002|interface=10.20.0.0/24
md.l2.streamId=2
md.l3.channel=aeron:udp?endpoint=239.10.0.3:40003|interface=10.20.0.0/24
md.l3.streamId=3
md.snapshot.channel=aeron:udp?endpoint=239.10.0.4:40004|interface=10.20.0.0/24
md.snapshot.streamId=4
md.snapshot.cycleMs=1000
```

The `interface=` qualifier is not optional on a multi-homed machine — without it Aeron picks an interface and it may not be the trading VLAN.

> **CONFIGURE IGMP SNOOPING AND A QUERIER ON THE VLAN**
> Without an IGMP querier, snooping switches age out group membership and the feed silently stops reaching subscribers, usually minutes after everything looked fine.

> **DO NOT CHANGE AERON'S FLOW CONTROL**
> It defaults to `MaxMulticastFlowControl` — the fastest receiver governs — and that is correct here. `MinMulticastFlowControl` would let the slowest subscriber throttle the publisher, reintroducing exactly the coupling that moving market data out of the engine was meant to remove. The accepted trade-off is that slow subscribers take unrecoverable gaps; gap detection and snapshot re-synchronisation are subscriber responsibilities, and `reference` provides both (`FeedSequenceTracker` , `DepthFeedAssembler`).

> **⚠ A FEED IS ONE CHANNEL**
> `market-data` cannot publish the same feed as both multicast and dynamic MDC, so a subscriber that cannot receive multicast cannot be served alongside one that can.

3.8 Process management

One `systemd` unit per process. The engine's is the template; the rest differ only in `ExecStart` and `CPUAffinity` .

```
# /etc/systemd/system/most-engine.service
[Unit]
Description=most-exchange matching engine (shard 0)
After=most-cluster-host.service
Requires=most-cluster-host.service

[Service]
Type=simple
User=most
ExecStart=/opt/most/bin/engine /etc/most/engine.properties
CPUAffinity=8
LimitMEMLOCK=infinity
LimitNOFILE=1048576

KillSignal=SIGTERM
TimeoutStopSec=60
Restart=on-failure
RestartSec=15s

[Install]
WantedBy=multi-user.target
```

`RestartSec=15s` is not arbitrary: it is longer than the roughly ten-second mark-file liveness timeout, so an automatic restart does not fail on `active mark file detected` and then back off.

> **THE WHOLE SHUTDOWN PATH HANGS OFF AN ORDERLY SIGTERM**
>
> Snapshot counters, latency summaries and the service container's own close all run inside a shutdown barrier that SIGTERM releases. `SIGKILL` skips them and leaves the mark file carrying a live timestamp, so the next start refuses for ten seconds. For native images `--install-exit-handlers` is load-bearing and is set in the root build; without it SIGTERM kills the image outright.

Configuration management

Two kinds of file that must not be confused:

- **Released artifacts** — `shard-N-securities.properties`, `shard-N-participants.properties`, the discovery registry. Published by the control plane as immutable numbered releases. Your configuration-management tool *fetches* a release and points `current` at it; it never authors one.
- **Node-local config** — `engine.properties`, `gateway.properties`, Aeron and cluster directories, feed channels. Lives in the repository and is templated per host.

3.9 Bringing a shard up

Order matters, and most of it is the mark-file timeout in disguise.

```
# 1. All three cluster hosts, within a few seconds of each other. A member that starts minutes
#    late will catch up, but the group will not form until two are present.
systemctl start most-cluster-host          # on shard0-a, -b, -c

# 2. Wait for a leader. Not optional: the engine's service container attaches to a consensus
#    module that must already exist.
journalctl -u most-cluster-host -f | grep -m1 'Leader'

# 3. Engine service containers, all three.
systemctl start most-engine

# 4. The rest, on their assigned nodes.
systemctl start most-market-data           # all three; followers stay silent
systemctl start most-discovery             # all three is safe

# 5. Gateways: independent ones on their own machines, co-located ones on every core machine.
#    Order does not matter; a co-located gateway stands by until its node leads.
systemctl start most-gateway

# 6. Control plane last — it is a client of everything above.
kubectl -n most rollout status deploy/control
```

Verify geometry before trusting anything

```
ansible shard0 -a "journalctl -u most-engine -n 200 --no-pager" | grep -i fingerprint | sort -u
```

One distinct value for `fingerprint` and one for `engineFingerprint`, or stop and fix it. Raft will not catch this, but the engines now do. Each one checks the leader's announced configuration in the log, and a node that disagrees prints `matching-engine: refused to go on` with both sets of values and exits non-zero. With a misconfigured *leader*, the healthy followers are the ones that stop. So read the values here before the first order rather than learn them from a refusal.

Establish the session baseline

A cold cluster has emitted no `SessionChanged`, so its phase is genuinely unknown to anything watching the feed. The scheduler **waits** rather than assuming `CLOSED`, because a control-plane restart mid-session looks identical from the outside and assuming would shut a live market.

Issue one session command deliberately as the last step of bringing a cold shard up. After that, the feed keeps the control plane current and the calendar takes over.

3.10 Blocking gaps

An honest list. None of these are large; all of them block a production deployment.

#	Gap	Blocks
~~1~~	~~`most-cluster` hardcodes member id 0~~ — **done**, `--member-id`	—
~~2~~	~~Driver and archive threading modes are hardcoded `SHARED`~~ — **done**, `--driver-threading`, `--archive-threading`	—
3	Idle strategies are constants, not configuration	3.4 — busy-spin only where cores are isolated
4	`engine.march` is x86-only and passed straight to `native-image`	A second architecture
5	The directory advertises one gateway per shard	3.2 — naming several gateways

#	Gap	Blocks
6	A feed is one channel; multicast and MDC cannot coexist	3.7 — mixed subscribers
7	No health endpoint on the core processes	6.8 — mechanised alerting; today everything is log lines and shutdown counters
8	None of this has been run across machines; three members on one machine have	All of section 3

4. Configuration in full

This section is a reference. Every key each process reads, every artifact a release contains, and every environment variable the control plane honours.

4.1 How a process is configured

Each of the four core processes takes **one positional argument**: the path to a properties file.

```
engine       /etc/most/engine.properties
gateway      /etc/most/gateway.properties
market-data  /etc/most/market-data.properties
discovery    /etc/most/discovery.properties
```

After the file is read, **JVM system properties in the same namespace override individual keys**. The prefix is the process's own: `engine.` , `gateway.` , `md.` , `discovery.` . This is how a deployment overrides one value without templating a whole file:

```
engine -Dengine.metrics=true /etc/most/engine.properties
```

The cluster host is different: it is a subcommand of the operator CLI and takes flags, not a file. The control plane is a Spring Boot application and takes environment variables.

> **A MISSING REQUIRED KEY STOPS THE PROCESS, AND THAT IS THE DESIGN**
>
> `engine.securitiesFile` and its equivalents have no default. Nothing in the shard security file has a default either — a wrong tick size misprices every order in silence, and a missing ISIN would be published to every downstream adapter.

4.2 The shard security file

The single most important artifact. It is read at boot by the engine, the gateway, market-data and discovery **alike**, so a shard has one definition and one fingerprint rather than four lists kept in step by hand. In production it is published by the control plane; the path each process is given should be a `current` symlink into a release directory.

```
shard.id=0
shard.securities=1,2

security.1.symbol=AAPL
security.1.isin=US0378331005
security.1.name=Apple Inc.
security.1.currency=USD
security.1.priceFloor=0
security.1.tickSize=1000000
security.1.levelCount=32768
security.1.maxOrders=1000000
```

Key	Type	Constraint
`shard.id`	int	Non-negative
`shard.securities`	int list	1 to 10 ids, comma-separated, no duplicates
`security.N.symbol`	string	Non-blank, at most 16 characters — the wire field's length
`security.N.isin`	string	12 characters, **check-digit validated at boot**
`security.N.name`	string	At most 48 characters
`security.N.currency`	string	Exactly 3 characters
`security.N.priceFloor`	int64	The price of ladder level 0, fixed point, 8 decimals
`security.N.tickSize`	int64	Positive. `1000000` is 0.01
`security.N.levelCount`	int	Positive. Ladder levels allocated per side
`security.N.maxOrders`	int	Positive. Order pool high-water mark; up to 1,000,000

LADDER GEOMETRY

`priceFloor`, `tickSize` and `levelCount` define a flat array of price levels per side, indexed by ticks above the floor. The ladder's range is `priceFloor` to `priceFloor + tickSize × levelCount`. It must be configured **strictly wider than the static circuit-breaker band**, so band rejection always fires before the ladder can overflow.

GEOMETRY CHANGES ARE ENFORCED AGAINST THE SNAPSHOT

On restart the engine reconciles this file against the snapshot it is restoring and **refuses to start** rather than lose state: a security removed while it holds resting orders, a changed `priceFloor`, `tickSize`, `levelCount` or `maxOrders`, an order that falls outside the new ladder, or resting-order counts that do not add up. A refusal is the system working. The two legitimate cases are quiet — an *empty* book leaving the shard, and a new security joining — and each is logged.

Apply a geometry change from `most cluster shutdown`, never after a SIGTERM. At the start of every leadership term the leader writes its configuration into the log, and every engine checks it. The log written since the last snapshot was written under the old file, and an engine that replays it under a new one refuses with `matching-engine: refused to go on` and exits non-zero. `most cluster shutdown` takes a snapshot at the end of the log, so there is nothing left to replay. Change the file on every node at once; a rolling geometry change is refused by construction.

Reference prices and collar widths are deliberately **not** in this file. They change during a session and every node must apply them at the same log position, so they arrive as `SecurityDefinition` commands through the replicated log (5.3).

4.3 The participant registry

Who may connect, and what each identity may do. Read by **three** processes for three reasons: the consensus module authenticates every cluster session against it and authorises snapshot requests from it, the engine resolves each session's principal into the participants it routes reports to, and the **gateway enforces it** on every client message.

```
shard.id=0
registry.gateways=control,gw-0,gw-1

gateway.gw-0.secret=3b2c637a03bcb9c167c9bd5f84de22f6bbdc18169d36526cafc164cd68725367
gateway.gw-0.participants=7,8,20,21,22,23
gateway.gw-0.cancelOnly=9
gateway.gw-0.operator=true

gateway.gw-1.secret=<sha-256 hex>
gateway.gw-1.participants=7,14

gateway.control.secret=310c47787a741d2f320f6e921645a52f305b5446acc38a9f6257b2a39cfd94a8
gateway.control.operator=true

participant.7.primary=gw-0
```

Key	Constraint
`shard.id`	Must match the shard of the process reading it, or the process refuses to start
`registry.gateways`	At least one id, comma-separated
`gateway.<id>.secret`	The **SHA-256 hex digest** of the shared secret, 64 lowercase hex characters
`gateway.<id>.participants`	Participants that may **place and cancel** through this gateway. Required unless the gateway is an operator
`gateway.<id>.cancelOnly`	Optional. Participants that may **cancel but not place** — revocation made graceful. Not also in `participants`. An operator can withdraw what such a participant leaves resting with `most cancel-all` (5.3), once this is published
`gateway.<id>.operator`	Optional, `true` or `false` (default). May send operator commands, and may request a snapshot through consensus. Any other value is refused
`participant.<id>.primary`	Required when a participant is listed on **more than one** gateway, and refused otherwise. Names the gateway it is bound to while both are connected; that gateway must list it

Gateway ids are at most 64 characters from a restricted alphabet. The file holds the digest, never the secret; the secret itself is delivered only to the process that presents it. A registry that uses none of the optional keys renders and fingerprints exactly as it did before they existed.

PARTICIPANT BINDING

A gateway presents `gatewayId:secret` as its cluster credentials. The consensus module verifies them and stamps the gateway id on the session as its **encoded principal**. Aeron carries that principal in the session-open event through the replicated log, so every node derives the identical participant-to-session map without any snapshot state of its own. A session binds each participant its gateway lists **if it is that participant's primary or nobody live holds it**, and a closing session's participants move to another open gateway that lists them, primary first. This is what makes a participant's fills deliverable when it has said nothing since its gateway last connected.

OPERATOR IDENTITY

An entry with `operator=true`. With no participants at all — the only kind of entry allowed to list nobody — it is how the **control plane and the CLI** are named to the cluster. A gateway may be an operator as well as speak for participants; the dev stack's `gw-0` is both.

What the gateway enforces

`UNAUTHORIZED_PARTICIPANT` is raised by the **gateway**, locally, before anything reaches the cluster:

- an order for a participant not in its `participants` — a `cancelOnly` participant included;
- a cancel for a participant in neither list. Checked, not optional: the engine's own cancel check is that the participant matches the order's, which only means something once the gateway has established the participant id is one this endpoint may use;
- an operator command — a session transition, purge, definition or image request, or any message it does not recognise — through a gateway that is **not** an operator is consumed and counted `refusedCommands`. There is no client report for it, so the counter is the only trace.

The client-to-gateway leg is not authenticated, so what this establishes is that *whoever can reach this gateway's endpoint may act for these participants*. Authenticating an end client is the job of the FIX and session gateways upstream; a member's connection lands on the gateway assigned to it.

The engine never rejects on the registry — a refusal decided from a file each node reads on its own schedule would diverge the nodes. It **counts** an order or cancel from a gateway that does not list the participant (`undeclaredParticipantMessages`), as defence against a misconfigured gateway.

> **A NODE WITH A REGISTRY AUTHENTICATES EVERYTHING OR NOTHING**
>
> A gateway with a bad secret fails to connect; it is never quietly downgraded to an anonymous session. And a client presenting **no** credentials is refused too: an anonymous session reaches the engine without passing any gateway, so it could act for any participant and send any operator command. The control plane (4.9) and the CLI (`--identity`, 5.6) therefore hold operator identities. A node started without a registry accepts everyone, as before the registry existed.

Reloading while running

The consensus module, the engine **and the gateway** re-read this file while running. They compare content by fingerprint rather than mtime — a release is published to a new directory and put in force by moving a symlink, so mtime says nothing — and swap an immutable registry behind a volatile reference. The gateway prints its own grants on every swap (`gateway: now participants=[…] cancelOnly=[…] operator=…`), which is the line to wait for when revoking someone.

A file that cannot be parsed, or one for another shard, is **reported and ignored**; the registry in force keeps applying. Standing down on a bad file would turn a typo into a shard that authenticates nobody. Watch `registryReloadFailures` rather than assuming a silent success.

4.4 Engine

```
engine.securitiesFile=/etc/most/current/shard-0-securities.properties
engine.participantRegistry=/etc/most/current/shard-0-participants.properties
engine.aeronDir=/dev/shm/aeron-most
engine.clusterDir=/var/lib/most/cluster
engine.bookEvent.channel=aeron:ipc
engine.bookEvent.streamId=12
```

Key	Default	Meaning
`engine.securitiesFile`	*required*	The shard security file (4.2)
`engine.participantRegistry`	none	The participant registry (4.3), for routing reports and counting `undeclaredParticipantMessages`. Omit and routes are learned from traffic alone
`engine.participantRegistry.reloadMs`	`5000`	Registry poll interval; `0` disables reloading

Key	Default	Meaning
`engine.aeronDir`	driver default	Must be the media driver this node's processes share
`engine.clusterDir`	`cluster`	The consensus module's cluster directory, written by the cluster host
`engine.serviceId`	`0`	The clustered service id
`engine.bookEvent.channel`	`aeron:ipc`	Where book events are published. Keep it IPC: market-data is on this node
`engine.bookEvent.streamId`	`12`	
`engine.auction.maxPasses`	`64`	Safety valve on the uncross fixed-point loop, not part of the algorithm. Can change an uncross result, so it is the **engine fingerprint** and must be identical on every node
`engine.backpressure.alertThreshold`	`1000000`	Consecutive back-pressured publications before alerting
`engine.reportRetention`	`1048576`	Execution reports every node keeps for a participant's resend request (128 B each since schema version 8, so ~156 MB at the default with its indexes). Must cover a failover's worth of reports, or a resend answers `TRUNCATED`. In the **engine fingerprint**: identical on every node (5.4, "After a failover")
`engine.metrics`	`false`	Hot-path timing; two clock reads per message. Also publishes the service thread's duty cycle as the `duty-ns: engine service` counter (5.8)
`engine.metrics.stages`	`false`	Adds the admit/match/settle partition of a new order; two more clock reads
`engine.metrics.file`	none	Where percentile distributions are written at shutdown
`engine.idleStrategy`	`busyspin`	How the service thread waits for work: `busyspin`, `backoff`, `yielding`, `sleeping` or `sleeping:<µs>`. Leave the default on an isolated core (3.4)

METRICS ARE NODE-LOCAL AND DELIBERATELY OUTSIDE THE FINGERPRINT

The engine reads `System.nanoTime()` to record these histograms, which the determinism rules otherwise prohibit. The rule is that time must not influence **replicated state**, not that it may not be observed: the histograms are write-only — never read by a branch, never snapshotted, never on a feed. Enabling metrics on one node and not another must be incapable of changing the log, the books or a snapshot. That is why `engine.metrics*` is excluded from the fingerprint, and it is the test any probe added later must pass. `engine.idleStrategy` is outside it too: how a node waits for work cannot change what it computes, so two members may differ.

4.5 Gateway

```
gateway.securitiesFile=/etc/most/current/shard-0-securities.properties
gateway.participantRegistry=/etc/most/current/shard-0-participants.properties
gateway.gatewayId=gw-a
gateway.credentialTokenFile=/etc/most/gw-a.secret

gateway.aeronDir=/dev/shm/aeron-gw-a
gateway.ingressChannel=aeron:udp
gateway.ingressEndpoints=0=shard0-a:20110,1=shard0-b:20110,2=shard0-c:20110
gateway.egressChannel=aeron:udp?endpoint=gw-a-host:0

gateway.client.inbound.channel=aeron:udp?endpoint=0.0.0.0:20001
gateway.client.inbound.streamId=20
gateway.client.outbound.channel=aeron:udp?endpoint=239.10.0.5:20002
gateway.client.outbound.streamId=21
```

Key	Default	Meaning
`gateway.securitiesFile`	*required*	The shard security file. Orders for anything else are rejected
`gateway.participantRegistry`	none	The registry this gateway **enforces** (4.3), and checks its own id against at boot
`gateway.participantRegistry.reloadMs`	`5000`	Registry poll interval; `0` disables reloading
`gateway.gatewayId`	none	This gateway's registry id. Unset means an anonymous connection that enforces nothing — and that a node with a registry refuses
`gateway.credentialToken`	none	The shared secret as a literal. Prefer the file form
`gateway.credentialTokenFile`	none	The shared secret from a file; trailing whitespace trimmed
`gateway.aeronDir`	driver default	This gateway's own media driver
`gateway.ingressChannel`	`aeron:udp`	Cluster ingress
`gateway.ingressEndpoints`	none	Every member's ingress endpoint, so the client can find the leader
`gateway.egressChannel`	`aeron:udp?endpoint=localhost:9020`	Cluster egress. The host must be one the cluster can **send to** — the name of the machine whose media driver this gateway uses — never `0.0.0.0`. `aeron:ipc` when `gateway.aeronDir` is the *leader's own* media driver: the highest-throughput setting (4.8, "The egress channel")
`gateway.client.inbound.channel`	`aeron:ipc`	Where adapters **publish orders**. Must be unique per gateway
`gateway.client.inbound.streamId`	`20`	
`gateway.client.outbound.channel`	`aeron:ipc`	Where the gateway **publishes execution reports**
`gateway.client.outbound.streamId`	`21`	

Key	Default	Meaning
`gateway.metrics`	`false`	Hot-path timing on both legs, and the poll thread's duty cycle as the `duty-ns: gateway <id>` counter (5.8)
`gateway.metrics.file`	none	Percentile distributions at shutdown
`gateway.idleStrategy`	`busyspin`	How the poll thread waits for work; same values as `engine.idleStrategy`
`gateway.placement`	`independent`	`independent`: its own tier, UDP to every member. `colocated`: one per node on that node's media driver (`gateway.aeronDir`), IPC both ways, active only while the node leads (3.2). Co-located needs `gateway.gatewayId`, and refuses a UDP ingress or egress channel and `gateway.ingressEndpoints`
`gateway.leaderPollMs`	`10`	How often a co-located gateway reads its node's role

`gateway.gatewayId` and one of the two credential keys must be set **together**: an id with no secret cannot authenticate, and a secret with no id has nothing to authenticate as. The process refuses to start otherwise, as it does when the id is absent from the registry or the registry is for another shard.

> **THE GATEWAY HAS NO STORAGE CONFIGURATION**
> It holds no per-order state. `origQty` and `cumQty` are stated by the engine on every execution report, so a gateway can be restarted or replaced at will and several can serve one shard. If you are migrating a configuration that sets `gateway.journalFile` or `gateway.journalSlots`, delete those keys; they no longer exist.

4.6 Market data

```
md.securitiesFile=/etc/most/current/shard-0-securities.properties
md.aeronDir=/dev/shm/aeron-most
md.bookEvent.channel=aeron:ipc
md.bookEvent.streamId=12
```

Key	Default	Meaning
`md.securitiesFile`	*required*	Depth geometry comes from here, so ladder level N in the engine is depth level N here by construction
`md.aeronDir`	driver default	The node's shared media driver
`md.bookEvent.channel` / `.streamId`	`aeron:ipc` / `12`	In from the engine
`md.l1.channel` / `.streamId`	`aeron:udp?endpoint=239.10.1.1:40001` / `1`	Top of book and last trade
`md.l2.channel` / `.streamId`	`aeron:udp?endpoint=239.10.1.2:40002` / `2`	Aggregated depth increments
`md.l3.channel` / `.streamId`	`aeron:udp?endpoint=239.10.1.3:40003` / `3`	Book events forwarded verbatim

Key	Default	Meaning
`md.snapshot.channel` / `.streamId`	`aeron:udp?endpoint=239.10.1.4:40004` / `4`	The L2 recovery feed
`md.snapshot.cycleMs`	`1000`	How long a full pass over the shard's books takes: the cycle is sliced one security per slice, so a joining subscriber waits one cycle however many securities there are
`md.metrics`	`false`	Publishes the poll thread's duty cycle as the `duty-ns: market-data` counter (5.8). Market data's only metric
`md.idleStrategy`	`busyspin`	How the poll thread waits for work; same values as `engine.idleStrategy`. At ~13% busy at full load, the first candidate for `backoff` where cores are short

L1, L2, L3

L1 is top of book plus last trade. **L2** is aggregated depth per price level, published as increments carrying absolute per-level quantities. **L3** is the engine's raw book event stream forwarded byte for byte — per-order detail, and the **only** feed on which a volatility halt is visible. A depth subscriber cannot distinguish a halt from a scheduled close, which is why the control plane subscribes to L3.

4.7 Discovery

```
discovery.shards=0
discovery.shard.0.securitiesFile=/etc/most/current/shard-0-securities.properties
discovery.shard.0.orderEntryChannel=aeron:udp?endpoint=gw-a:20001
discovery.shard.0.orderEntryStreamId=20
discovery.shard.0.executionReportChannel=aeron:udp?endpoint=239.10.0.5:20002
discovery.shard.0.executionReportStreamId=21

discovery.aeronDir=/dev/shm/aeron-most
discovery.channel=aeron:udp?endpoint=239.10.0.1:40000
discovery.streamId=100
discovery.intervalMs=2000
```

Key	Default	Meaning
`discovery.shards`	*required*	Shard ids to publish
`discovery.shard.N.securitiesFile`	*required*	The shard's own security file, so the directory and the running engine cannot describe different geometry
`discovery.shard.N.orderEntryChannel` / `StreamId`	*required*	The **gateway's** client inbound endpoint
`discovery.shard.N.executionReportChannel` / `StreamId`	*required*	The gateway's client outbound endpoint, in **subscriber** form
`discovery.aeronDir`	driver default	
`discovery.channel` / `.streamId`	`aeron:udp?endpoint=239.10.0.1:40000` / `100`	The broadcast

Key	Default	Meaning
`discovery.intervalMs`	`5000`	Broadcast cycle

> **THE DIRECTORY PUBLISHES GATEWAY ENDPOINTS, NEVER CLUSTER INGRESS**
>
> An adapter that connected to the cluster directly would bypass the gateway's `securityId` validation and the participant binding that decides where a maker's fills go. Channels are limited to 128 characters, which is the wire field's length.

Discovery enforces **one shard per security** across the whole universe and refuses to start otherwise.

4.8 The cluster host

`most cluster` takes flags rather than a properties file.

Flag	Default	Meaning
`--dir DIR`	`build/cluster-host`	Base directory; the three below default beneath it
`--aeron-dir DIR`	`<dir>/driver`	Media driver directory
`--archive-dir DIR`	`<dir>/archive`	Archive directory — **durable**
`--cluster-dir DIR`	`<dir>/cluster`	Consensus module directory — **durable**
`--host HOST`	`localhost`	Host name used to build default endpoints
`--members STRING`	single-node default	Aeron member string (3.6)
`--member-id N`	`0`	Which entry of `--members` this node is
`--ipc-ingress`	off	Let a gateway on this node's media driver reach the consensus module over IPC while it leads. Needed by `gateway.placement=colocated`
`--participants FILE`	none	The participant registry (4.3). With it, every cluster session must authenticate — anonymous ones are refused — and a snapshot request through consensus is granted only to an `operator=true` identity. Without it, anyone connects and anyone may snapshot
`--participants-reload-ms N`	`5000`	Registry poll interval; `0` disables
`--driver-threading MODE`	`SHARED`	Media driver threads: `SHARED`, `SHARED_NETWORK` or `DEDICATED`. **The shard's first throughput ceiling**; the gateway's egress channel is the second (see below)
`--archive-threading MODE`	`SHARED`	Archive threads: `SHARED` or `DEDICATED`. Leave it alone unless the cores are isolated
`--ingress-term-length LEN`	`64k`	Cluster ingress term length, e.g. `16m`. Buys latency headroom at the edge, not capacity
`--duty`	off	Publish a duty-cycle counter for every thread this process runs — the driver's (per threading mode), the archive's and the consensus module's (5.8). Measures around Aeron's idle strategies without changing them
`--fresh`	off	**Delete the archive and cluster directories on start**

Flag	Default	Meaning
`--keep`	on	Persist them. Contradicts `--fresh`

What caps a shard's throughput

Two settings decide what a shard can carry, and they bind in order: the media driver's threading mode, then the gateway's cluster egress channel. Everything below was measured on one node, with ten securities, and gives the aggregate rate across all of them.

Driver threading. `SHARED` puts the media driver's conductor, sender and receiver on **one** thread. At ten securities that thread moves ~190 MB/s of loopback traffic and is what caps the shard first. Measured on a 14-core development machine:

`--driver-threading`	Sustained	p50 at 350,000/s
`SHARED` (default)	~350,000 orders/s	6,410 µs
`DEDICATED`	**~550,000 orders/s**	**79 µs**

1.6x the throughput and 81x the median at the edge. **The default is `SHARED` on purpose:** `DEDICATED` busy-spins three threads, which starves a development machine that is already running five JVMs, and it needs the isolated cores of 3.4 to be worth having. So:

- **Development** — leave both defaults alone. 2.3 and the e2e scripts are correctness checks, not capacity ones.
- **Performance measurement and production** — `--driver-threading DEDICATED`, with the core allocation of 3.4 in place.

The egress channel. With dedicated driver threads, the next cap is the stream that carries execution reports back to the gateway. Each order produces about 2.2 of them, ~350 bytes of egress against ~128 bytes of ingress. Over UDP, Aeron's sender puts out **at most one datagram of the channel's MTU (1,408 bytes by default) per publication per pass**. So egress is limited by datagrams per second, not by bytes. When it overflows, the engine's egress publication fills and the engine waits on it, and waiting reads as 100% busy (5.8). Measured on the same machine, same day:

`gateway.egressChannel`	Sustained	p50 at 650,000/s
`aeron:udp?endpoint=host:0` (1,408 B datagrams, the default shape)	~500,000 orders/s	441 ms (a queue)
`aeron:udp?endpoint=host:0|mtu=8192` (8 KB datagrams)	at least 650,000/s; higher not measured	82 µs
`aeron:ipc`	**~1,500,000 orders/s**; 1,000,000/s held at an 83 µs median	66–72 µs

`aeron:ipc` **is the highest-throughput setting there is**, and above ~1,500,000/s it's the engine's own thread that fills (5.7). It has one requirement: **the gateway must use the leader's media driver**, with `gateway.aeronDir` pointing at that node's driver directory. IPC does not cross from one media driver to another. The development stack and the e2e scripts already run their gateway on the shard's driver. So:

- **A single-node shard** (development, performance measurement): `gateway.egressChannel=aeron:ipc`. It always holds, because the one node always leads. The scripts take it as `EGRESS_CHANNEL=aeron:ipc`.
- **A multi-node cluster** chooses with `gateway.placement` (3.2). `colocated` keeps IPC egress across failovers by running a gateway on every node, active only on the leader's, with clients moving between them. `independent` uses UDP egress, where fewer and larger datagrams (`|mtu=8192`) are the lever that removed

the same step on loopback. On a real network an MTU above the link's needs jumbo frames end to end on the trading VLAN, and that combination has not been measured. Neither placement's *throughput* has been measured multi-node: log replication to followers crosses the same UDP sender either way.

> **A 100% ENGINE IS NOT ALWAYS A BUSY ONE**
> The engine retries a full publication inside its work, so behind a full egress publication it reads 99.9% busy and its `admit` and `match` stages look dearer, while doing nothing useful. Before concluding that the engine is the limit, read the egress publication's headroom (5.8, "A publication at its limit").

> **DO NOT DEDICATE THE ARCHIVE'S THREADS ON THEIR OWN**
> `--archive-threading DEDICATED` behind a `SHARED` driver is measurably **worse than both shared** — 28.4 ms against 6,410 µs at 350,000/s, and saturated where the default was not — because it takes a core from the component that needed it. It belongs with dedicated driver threads and isolated cores, or not at all. `--ingress-term-length` is the same shape of trap: raising it from `64k` to `16m` improves p50 at the edge 2.9x and moves the sustainable rate **not at all**.

> **WHAT "SUSTAINED" MEANS HERE**
> The figures above are the rate at which acknowledgements keep up. Above it the shard still accepts everything — nothing is rejected and nothing is dropped — and the latency becomes a queue that never drains: at 1,000,000 orders/s offered with UDP egress, every order was still answered, at a median of 462 ms. A rate above the knee therefore reports an *offer* rate and not a throughput, which is why `e2e/run-sweep.sh` marks such a row `SATURATED` (5.7).

> **THE ARCHIVE AND CLUSTER DIRECTORIES ARE THE SHARD'S ONLY RESUMPTION POINT**
> They persist by default and `--fresh` is how you ask to lose them. The **Aeron directory is different** and is always recreated: it is memory-mapped IPC buffers and a `cnc.dat`, and keeping it makes a restart fail with `Active media driver detected` until the previous driver's liveness timeout expires.

4.9 The control plane

Configured entirely by environment variables.

Database and web

Variable	Default	
`CONTROL_DB_URL`	`jdbc:postgresql://localhost:5432/most_control`	Flyway migrates on start
`CONTROL_DB_USER` / `CONTROL_DB_PASSWORD`	`most` / `most`	
`CONTROL_PORT`	`8080`	
`CONTROL_RELEASE_DIR`	`./build/releases`	Where numbered release directories are written
`CONTROL_COOKIE_SECURE`	`false`	**Set `true` behind TLS**

Authentication

Variable	Default	
`CONTROL_ADMIN_USER`	`admin`	The first operator, seeded once into an empty table
`CONTROL_ADMIN_PASSWORD`	*unset*	**With no password set, one is generated and logged at WARN on first boot**

> **LEAVE `CONTROL_ADMIN_PASSWORD` UNSET OUTSIDE DEVELOPMENT**
> A known default password on something that can open a market would be worse than no authentication at all, because it would look protected. Passwords are stored as BCrypt hashes in `control_user`.

The live link to a running exchange

Variable	Default	
`CONTROL_AERON_ENABLED`	`true`	`false` for a database-only deployment
`CONTROL_AERON_DIR`	driver default	Must be a media driver this host can reach
`CONTROL_DISCOVERY_CHANNEL` / `_STREAM`	`aeron:udp?endpoint=239.10.0.1:40000` / `100`	The routing table
`CONTROL_L3_CHANNEL` / `_STREAM`	`aeron:udp?endpoint=239.10.1.3:40003` / `3`	Phase, trades and **halts**
`CONTROL_L2_CHANNEL` / `_STREAM`	`aeron:udp?endpoint=239.10.1.2:40002` / `2`	Depth increments for the console
`CONTROL_SNAPSHOT_CHANNEL` / `_STREAM`	`aeron:udp?endpoint=239.10.1.4:40004` / `4`	Depth images
`CONTROL_L1_CHANNEL` / `_STREAM`	`aeron:udp?endpoint=239.10.1.1:40001` / `1`	Last trade
`CONTROL_DEPTH_PUBLISH_MS`	`250`	How often a conflated book image is pushed to browsers
`CONTROL_DEPTH_MAX_LEVELS`	25	Depth per side in that image
`CONTROL_CLUSTER_OPERATORCHANNEL_<shardId>` / … `OPERATORSTREAM_<shardId>`	the shard's order-entry endpoint	Where operator commands go (below)

Operator commands go through a gateway that must be an operator. Session transitions, purges, definitions and image requests — the scheduler's included — are sent to a gateway like any client message, and only one whose registry entry has `operator=true` forwards them (4.3). They are unacknowledged, so a refusal is invisible here: the gateway consumes and counts it (`refusedCommands`). By default they go to the shard's advertised order-entry endpoint; when that gateway is a participant's rather than the operator's, name a dedicated operator gateway with `CONTROL_CLUSTER_OPERATORCHANNEL_<shardId>` and, optionally, `…OPERATORSTREAM_<shardId>`.

The link is **optional**. Authoring reference data must work with no media driver anywhere near it, so a missing driver is reported by `GET /api/status` and nothing else breaks.

L2, the snapshot feed and L1 go together: increments cannot be applied without an image, an image goes stale without the increments, and a synchronised subscriber ignores snapshots — so without L1 a book would trade on

screen while its last price stayed empty.

Cluster ingress, for snapshots

A snapshot request is a **cluster admin request**, not a message for the log, so it is the one thing the control plane does not send through a gateway.

Variable	Default	
`CONTROL_CLUSTER_INGRESS_CHANNEL`	`aeron:udp`	
`CONTROL_CLUSTER_EGRESS_CHANNEL`	`aeron:udp?endpoint=0.0.0.0:0`	**Set it.** The host must be one the cluster can send to — the machine of this process's media driver. With the default the cluster's answer has nowhere to go and every request times out
`CONTROL_CLUSTER_INGRESS_<shardId>`	none	Member endpoints, e.g. `0=shard0-a:20110,1=shard0-b:20110`
`CONTROL_CLUSTER_IDENTITY_<shardId>`	none	The registry identity to present. Required when the shard runs with a registry, which refuses anonymous sessions
`CONTROL_CLUSTER_SECRETFILE_<shardId>`	none	Its secret, from a file only. Set with the identity or not at all

The identity must be an **operator** entry of that shard's registry — `operator=true`, usually with no participants. The consensus module grants a snapshot request only to an operator (4.3), and the control plane reports **what the cluster answered**: `confirmed` means it answered OK, which it does once the snapshot has been taken; a refusal comes back with the cluster's message, and silence as a timeout. (Aeron's own default grants no snapshot request at all. Until the registry authorised operators, every snapshot asked for over the network was refused, and reported as confirmed because only the offer was checked.)

A shard with no ingress entry simply cannot be snapshotted from the control plane, and the API says so rather than failing obscurely. This is deliberately per-process configuration rather than database or directory data: keeping cluster ingress out of the broadcast directory is what stops an upstream adapter finding the cluster and skipping the gateway.

Scheduler

Variable	Default
`CONTROL_SCHEDULER_ENABLED`	`true`
`CONTROL_SCHEDULER_INTERVAL_MS`	`5000`

4.10 Authoring configuration in the control plane

The database holds a **draft topology** that nothing reads until it is published.

Table	Holds
`shard`	Shard id and the gateway client endpoints published in the directory
`security`	Identity and geometry (published), plus reference price and collar widths (sent as commands, never published)
`participant`	Participant id, name, SMP id, enabled

Table	Holds
`gateway`, `gateway_participant`	Gateway identity, secret digest, `operator`; and per listing, `cancel_only` and `is_primary`
`spec_release`, `spec_release_shard`	Published releases and the fingerprints frozen into each
`session_schedule`, `session_schedule_entry`, `market_holiday`	The trading calendar
`control_user`, `operator_audit`	Operators and who asked for every market-moving command

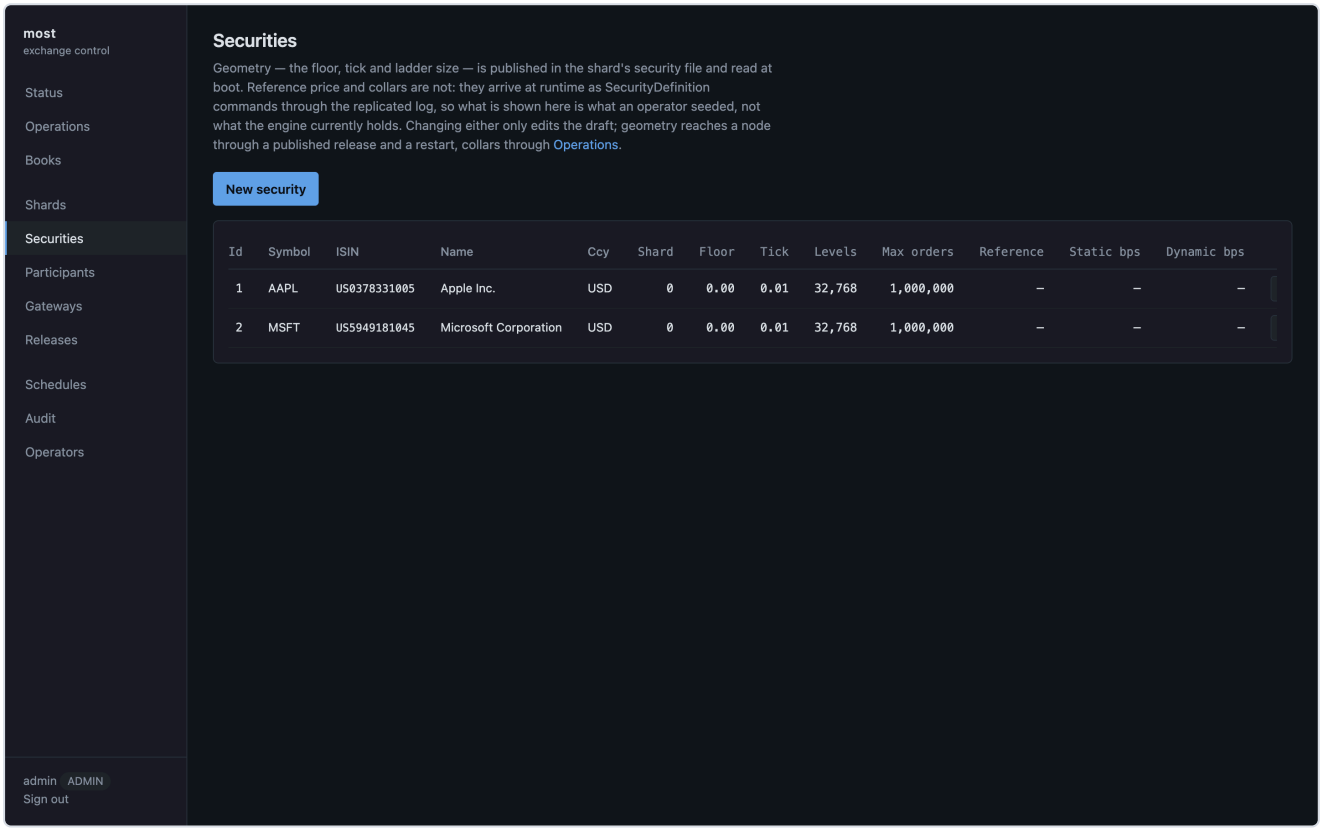


Figure 4.1 Securities. Identity and ladder geometry, validated by the same domain objects a process uses at boot — the ISIN check and the length limits have exactly one implementation.

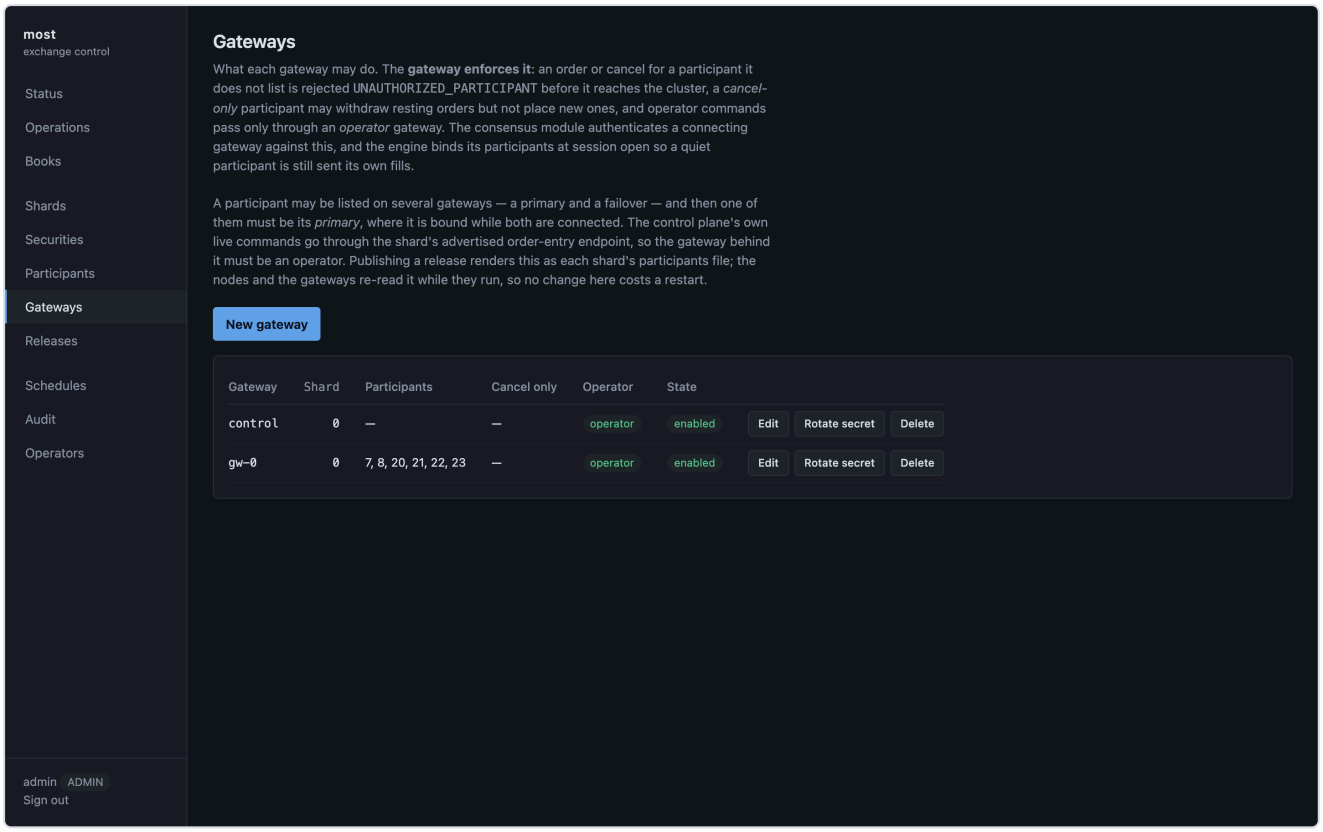


Figure 4.2 Gateways. What each identity may do: participants that place (P marks a primary), those that may only cancel, and whether it is an operator. Creating a gateway returns its secret exactly once.

A participant may be listed on several gateways; where several of a shard's gateways list it, one must be marked its primary, or the draft view reports the shard's registry as a problem and the release cannot be published. One primary per participant per shard is refused at the point of the mistake. An operator-only gateway (operator, no participants) is published like any other — it is the control plane's and the CLI's identity.

> **A GATEWAY SECRET IS RETURNED EXACTLY ONCE**
>
> `POST /api/gateways` generates one and returns the plaintext in that response and nowhere else; the database stores only its SHA-256. The digest cannot be BCrypt, because the cluster must reproduce it to verify what a gateway presents. Rotating with `PUT /api/gateways/{id}/secret` issues a new one — there is no way to read the old one back.

`GET /api/topology` reports **every** problem rather than the first, because a half-built topology is the normal state while someone is editing it. An empty `problems` list is exactly the condition for publishing, so one call answers both "can I publish?" and "what is stopping me?".

4.11 Publishing a release

```
curl -u admin:... -X POST localhost:8080/api/releases \
  -H 'Content-Type: application/json' -d '{"note":"add TSLA to shard 1"}'
```

A release directory contains the rendered artifacts every process boots from, plus a manifest recording each shard's geometry fingerprint and registry fingerprint separately.

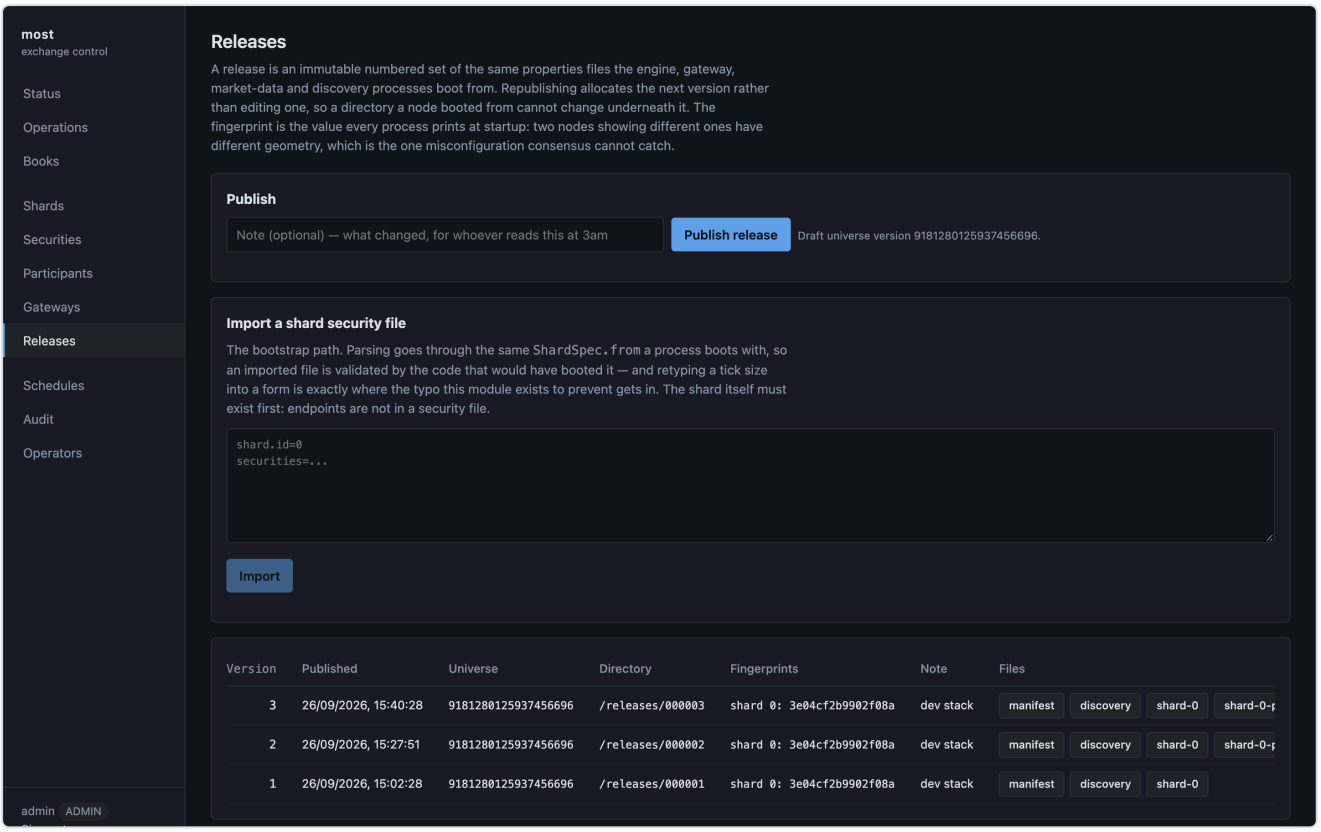


Figure 4.3 Releases. Immutable and numbered; the exact bytes of any artifact can be read back, and an existing shard security file can be imported to seed the draft.

> **RELEASES ARE IMMUTABLE**
>
> Republishing allocates the next version rather than overwriting. A release carries **topology only** — Aeron directories, cluster directories and feed channels stay in each process's own configuration.

Deploying a release is: fetch it to every machine, move the `current` symlink, and restart what needs restarting. What needs restarting depends on what changed — see 6.6.

> **⚠ PROCESSES DO NOT VERIFY THEY READ THE RELEASE YOU THINK THEY DID**
>
> Every process prints its fingerprint and the control plane records the fingerprint it published, but comparing them is the operator's job. Making the four processes fail fast on a mismatch changes four boot paths and is deliberately a separate change.

5. Daily operations, order flow and monitoring

5.1 The trading day

Time	Action	How
Before the open	**Purge expired orders**	`PurgeExpiredOrders`, its own sequenced command
Before the open	**Seed or re-seed reference prices and collars** if they have moved	`SecurityDefinition` per security
Open – 1 hour	`PRE_OPEN` — orders accepted and booked, nothing matches	`SessionTransition`
Open – 5 minutes	`OPEN_AUCTION` — still accepting, still not matching	`SessionTransition`
Open	`CONTINUOUS` — **the uncross runs on this transition**	`SessionTransition`
Close	`CLOSED` — orders rejected, the resting book stays	`SessionTransition`
After the close	**Take a snapshot**	Cluster admin request

All five commands can be issued from the CLI, from the REST API, from the admin console, or by the control plane's scheduler. In production the scheduler does all of it and an operator intervenes.

> **EXPIRY IS DATE-BASED ONLY**
> Orders carry `expireDate` as a `YYYYMMDD` integer, where `0` means good-till-cancelled, and are expired when `0 < expireDate < currentTradingDate`. There is no intraday expiry. The purge is its own sequenced command and must run **well before** `PRE_OPEN`: after `PRE_OPEN` the book is accepting orders and a late sweep would expire orders someone had just placed. The scheduler skips a late purge rather than running it.

5.2 The trading calendar

A **schedule** is a named trading day — local times mapped to phases, plus a purge time — that a shard follows. It lives in the control plane and fires there, not in the engine.

That costs no determinism: the `SessionTransition` it emits is sequenced through the replicated log like any other command, so every node applies it at the same log position. What it buys is that holidays and daylight saving stay outside the deterministic state machine, where a bug kills every node simultaneously.

```
curl -X PUT localhost:8080/api/schedules/equities -u admin:... \
  -H 'Content-Type: application/json' -d '{
  "name": "equities", "zone": "Europe/Amsterdam",
  "weekdays": ["MONDAY","TUESDAY","WEDNESDAY","THURSDAY","FRIDAY"],
  "purgeTime": "07:00",
  "entries": [
    { "at": "08:00", "phase": "pre-open" },
    { "at": "08:55", "phase": "open-auction" },
    { "at": "09:00", "phase": "continuous" },
    { "at": "17:30", "phase": "closed" } ] }'

curl -X PUT localhost:8080/api/shards/0/schedule -u admin:... \
  -H 'Content-Type: application/json' -d '{"scheduleName":"equities"}'

curl -X POST localhost:8080/api/schedules/equities/holidays -u admin:... \
  -H 'Content-Type: application/json' -d '{"date":"2026-12-25","description":"Christmas"}'
```

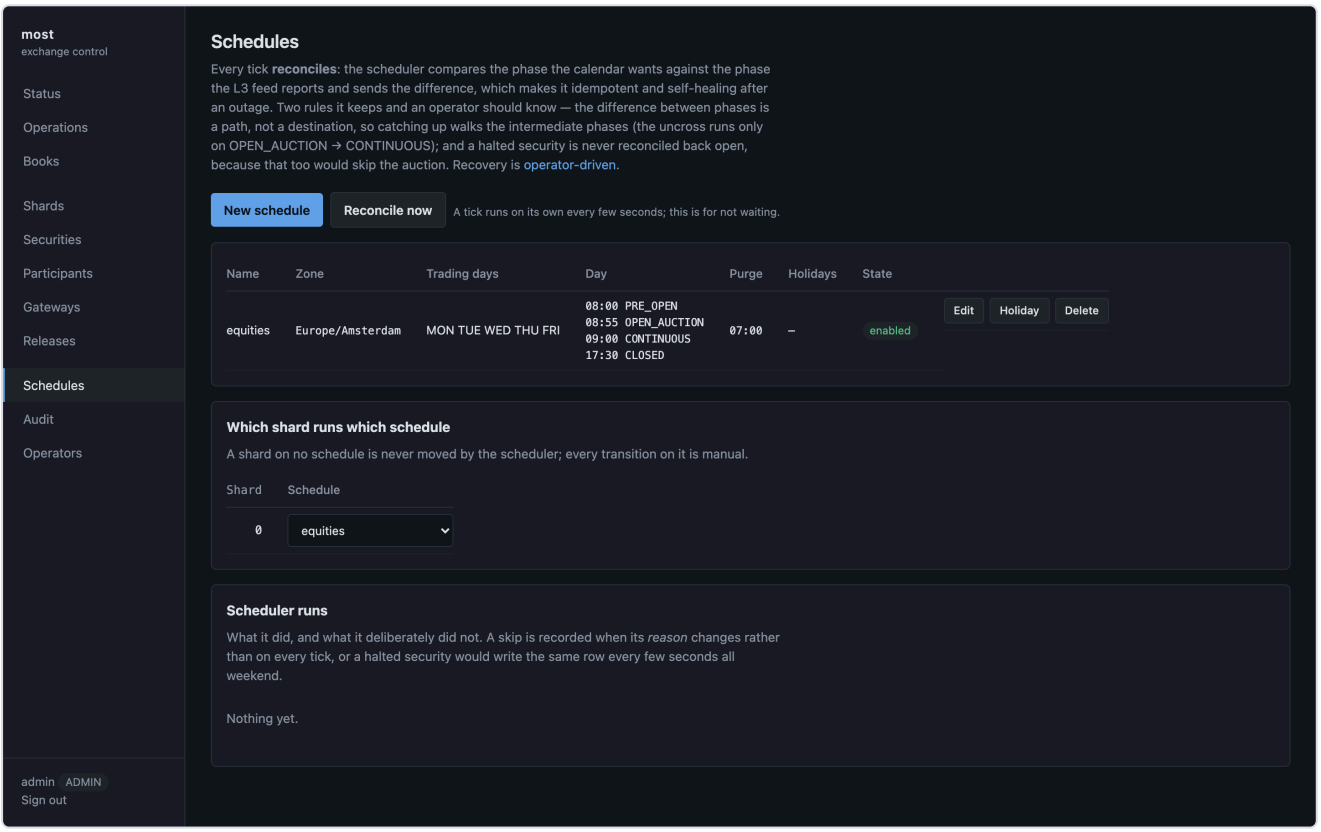


Figure 5.1 Schedules. The calendar, its holidays, which shard follows which schedule, and the reconciler's own log of what it did and what it refused to do.

Times are **local**, and the zone is stored with them: a market opens at 09:00 local whatever the offset is that week. Storing UTC instants would move the open by an hour twice a year.

Every tick is a reconciliation

The scheduler does not fire timers. On each tick it compares the phase the calendar says a shard should be in against the phase **the L3 feed says it is in**, and sends the difference. One choice, three consequences: it is idempotent, it recovers by itself after an outage, and there is no missed-timer state to keep anywhere.

THE DIFFERENCE BETWEEN TWO PHASES IS A PATH, NOT A DESTINATION

The uncross runs only on `OPEN_AUCTION → CONTINUOUS`. A control plane that was down through the open and catches up at 09:15 walks `PRE_OPEN → OPEN_AUCTION → CONTINUOUS`. Sending `CONTINUOUS` straight from `CLOSED` would be accepted by the engine and would **silently skip the auction**, leaving a crossed resting book crossed until an aggressor happened to arrive. Closing is a single step — nothing is computed on the way down.

What the scheduler refuses to do, and says so rather than doing quietly:

- **It never reopens a halted security.** Recovery is operator-driven by design, and reconciling back to `CONTINUOUS` would do it *without an auction*, since from the scheduler's point of view the shard is already past `OPEN_AUCTION`.
- **It waits rather than guessing an unknown phase.** See 3.9.
- **It skips a late purge** rather than running it.
- **It stands down** when a shard's books disagree on their phase, and when another control-plane instance holds its Postgres advisory lock. A duplicated transition is survivable; a duplicated purge is not something to leave to luck.

```
GET  /api/scheduler/runs                          what it did, and what it deliberately did not
POST /api/scheduler/tick                          reconcile now instead of waiting
GET  /api/schedules/{name}/preview?date=2026-12-25
```

A skip is recorded when its **reason changes**, not on every tick, so a halted security appears once rather than every five seconds all weekend.

5.3 Operator commands

Five commands reach a running exchange. `SecurityDefinition`, `SessionTransition`, `PurgeExpiredOrders` and `RequestBookImage` go to the **gateway's client channel** like any other message and are forwarded into the log untouched. A snapshot request is different: it is a cluster admin request and goes to cluster ingress.

OPERATOR COMMANDS ARE NOT ACKNOWLEDGED

The engine applies or rejects them without replying. A rejected `SecurityDefinition` increments a counter and says nothing else. A sender may therefore claim only that **bytes were sent** — never that a command was applied. The control plane separates `sent` from `confirmed`, and confirms a phase only by afterwards seeing `SessionChanged` on the L3 feed.

Seeding a definition

```
$ most define --symbol AAPL --reference 101.30 --static-collar 5000 --dynamic-collar 100
defined AAPL on shard 0: reference 101.30, static 5000bps, dynamic 100bps
```

STATIC AND DYNAMIC REFERENCE

`staticReference` is seeded here and is reset only by an **executing** uncross. Its collar gates order *acceptance*: an order outside the band is rejected with `PRICE_OUT_OF_BOUNDS`. `dynamicReference` is seeded the same way but tracks **every** trade, and its collar gates *execution* per price level. An uncross that produces no trade leaves both untouched.

Everything checkable is checked **before** the bytes go out, because nothing will report a rejection. The control plane refuses a definition whose static band falls outside the pre-allocated ladder:

```
a 5000bp static band around 300.00 spans 150.00..450.00, outside AAPL's ladder
0.00..327.67; the engine would reject it without replying
```

Geometry in a definition is never taken from the caller — `priceFloor`, `tickSize` and `levelCount` come from the same rows the shard booted from, because a definition whose geometry disagrees with the allocated book is rejected in silence.

Moving the session

```
$ most session --phase continuous --shard 0
shard 0 → continuous (trading date 20260906)
```

```
{ "command": "session CONTINUOUS shard 0", "sent": true, "confirmed": true,
  "detail": "every book on shard 0 reported CONTINUOUS" }
```

A SESSION TRANSITION IS SHARD-WIDE

The engine applies the phase to every book it hosts. There is no per-security session command, so moving one security moves every other book on that shard. The REST response says so every time, not only when it bites.

Purging expired orders

```
$ most purge --shard 0
shard 0 purged for trading date 20260906
```

The purge walks the **price ladders**, not the order id map — that map compacts its probe chain on removal, so iterator-based removal can silently skip entries.

Republishing books as an image

For a market-data process that restarted while the engine kept running, and therefore has no book and no way to learn one from a live feed:

```
$ most image --shard 0
shard 0 asked to republish its books
```

BOOK IMAGE

One message per **occupied ladder level** — bounded by `levelCount` rather than by a resting depth nobody has measured — bracketed by begin and end messages. It carries the current sequence number as a **baseline and consumes none**, so a subscriber must not count an image's `seqNum` as a gap. Installing an image publishes increments as well as a snapshot, because a subscriber that is already synchronised ignores snapshots.

A snapshot restore also sets this pending, so both triggers use one path: publication happens from background work once the book-event publication is *connected*, because both ask at the moment a subscriber is least likely to be listening.

Cancelling one participant's orders

The operator side of revocation (4.3, 6.7). It cancels every resting order of one participant on a shard, or on one security with `--symbol`:

```
$ most cancel-all --participant 42 --shard 0
participant 42: every resting order on shard 0 sent for cancellation; nothing acknowledges it --
each order leaves the book as an ordinary cancel
```

`--participant` is required. Unlike every other command, there is no default, because a default would make a forgotten flag cancel somebody's whole book. The control plane sends the same command from `POST /api/shards/{id}/participants/{pid}/cancel-orders` and from the Operations page. It goes through an **operator** gateway like the other operator commands, in every phase, a halt included.

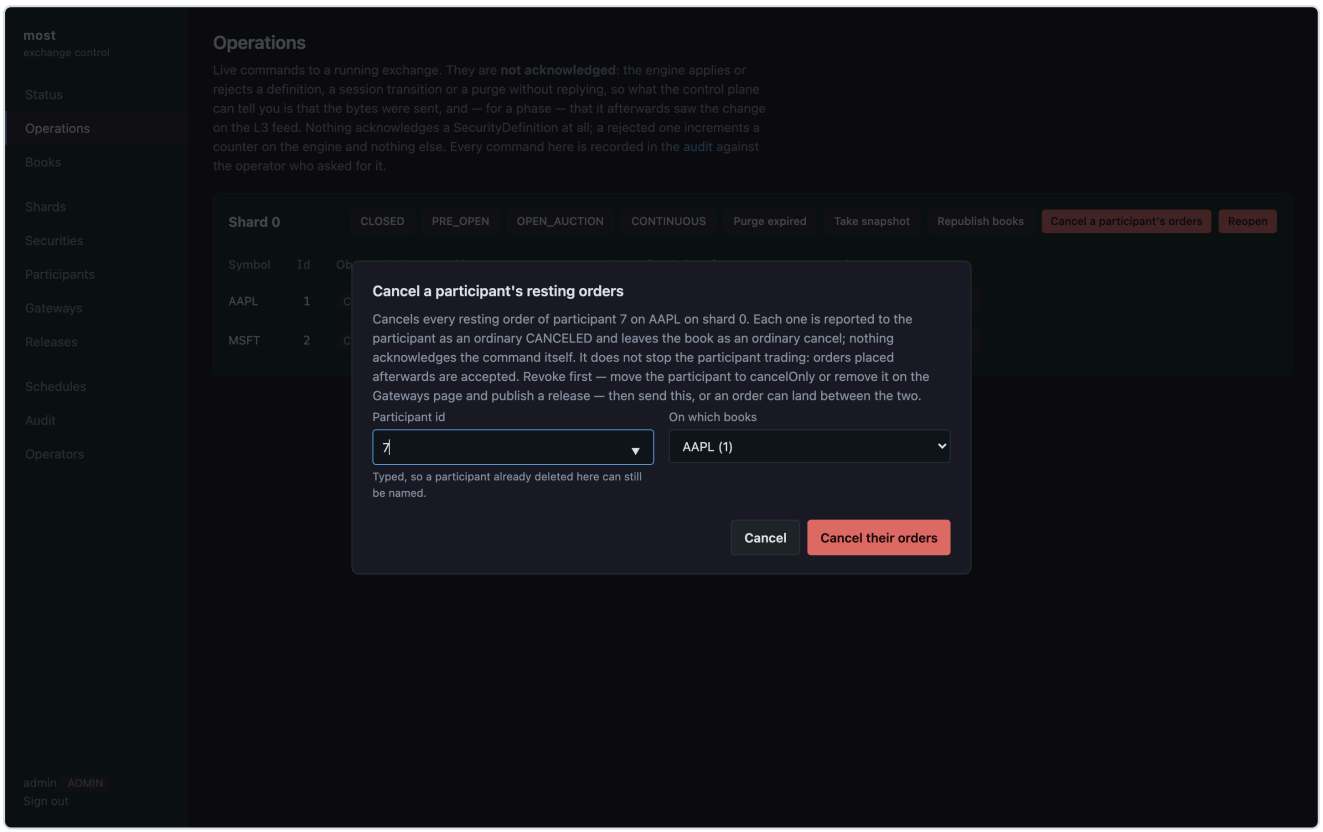


Figure 5.2 Cancelling one participant's orders from the Operations page. The participant id is typed rather than only picked, so a participant already deleted from the control plane can still be named. The confirmation states the scope and the order of the steps rather than asking whether you are sure.

The participant receives an ordinary `CANCELED` report for each order, carrying that order's own `clOrdId`. The market sees ordinary cancels: L3 does not say an operator did it, and nothing acknowledges the command. The engine counts `bulkCancelledOrders` at shutdown. **It decides nothing about the future.** Orders placed afterwards are accepted, so publish the revocation first.

5.4 Order flow

The order entry message

Field	Notes
`participantId`	The trading firm
`clOrdId`	The participant's own order id: 1 to 20 characters of `A-Z a-z 0-9 - _ + ; : . /`. Echoed on every report for this order. Unique per participant for the trading day; nothing checks that
`price`	Fixed point, 8 implied decimals

Field	Notes
`qty`	Fixed point, 8 implied decimals
`smpId`	Self-match prevention id; `0` means use `participantId`
`securityId`	Validated by the gateway against its shard's list
`expireDate`	`YYYYMMDD`, or `0` for good-till-cancelled
`side`	`BUY` or `SELL`
`smpStrategy`	`CANCEL_AGGRESSOR` (default) or `CANCEL_RESTING`

A cancel carries `participantId`, `origClOrdId`, a new `clOrdId`, the `exchangeOrderId` to cancel, `securityId` and `side`. It cancels only if its participant and its `origClOrdId` match the order's, all 20 bytes of the id.

Every execution report carries both `clOrdId` and `origClOrdId`. A cancel's report carries the cancel's own `clOrdId` and the order's as `origClOrdId`, and so does a cancel's rejection. Every other report — a new order's, a fill's, an expiry's, a bulk cancel's — carries the order's own id in both.

> **SCHEMA VERSION 8**
>
> The wire format since 2026-11-01. It made `clOrdId` the participant's own text, where it had been a 64-bit integer, and added `origClOrdId` to every report. The layout of orders, cancels and reports changed in place, so a process on an older version cannot be read by one on version 8: moving to it is a whole-shard upgrade (6.7).

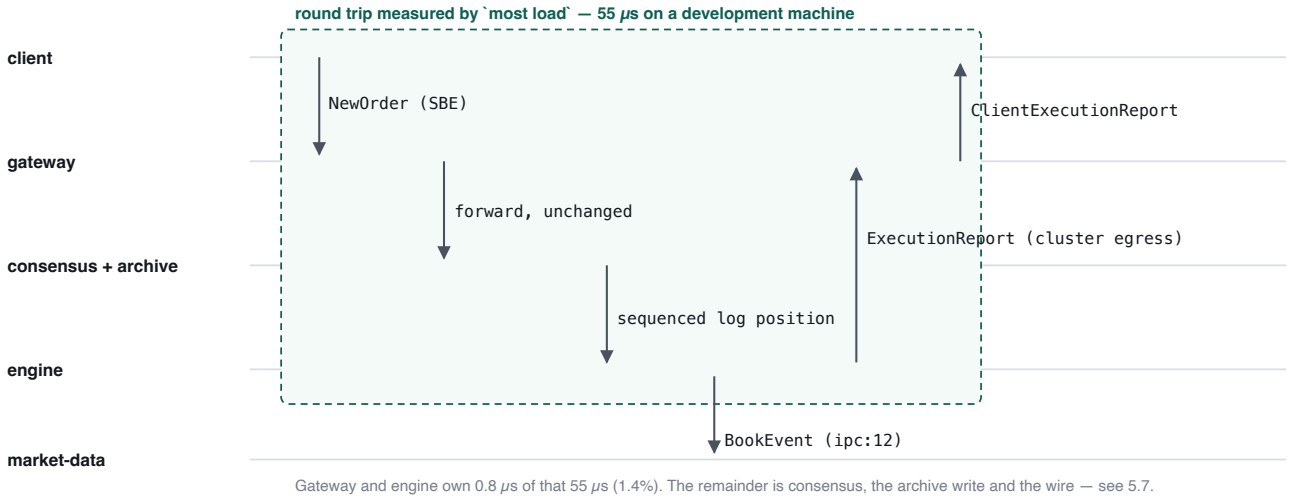


Figure 5.3 One client round trip through the stages that own it. The gateway and the engine are the two smallest.

What happens to an order

1. **Gateway.** The participant is checked against the gateway's registry entry, then the client order ids (`INVALID_CL_ORD_ID`), then `securityId` against the shard's list (`UNKNOWN_SECURITY`). A rejection here never reaches the cluster. The message is then forwarded into cluster ingress **unchanged**.
2. **Consensus.** The message is replicated and sequenced. Every node will apply it at the same log position.
3. **Engine — admit.** Phase gate (`MARKET_CLOSED` outside the four phases below `CLOSED`), static collar (`PRICE_OUT_OF_BOUNDS`), ladder range (`PRICE_OUT_OF_LADDER`), capacity (`BOOK_CAPACITY`), expiry (`ORDER_EXPIRED`).
4. **Engine — match**, only in `CONTINUOUS`. Per price level the dynamic collar is checked first, as a level gate; self-match prevention is checked inside the level, as an order gate.

5. **Engine — settle.** The remainder is booked, execution reports are generated, and book events are published on the leader's IPC stream.

> **AN ORDER THE GATEWAY CANNOT FORWARD IS NOT CONSUMED**
>
> A cluster offer returning back-pressure is **transient**, and treating it as a drop would lose an order the client believes it placed, with no acknowledgement and no rejection. The gateway leaves the fragment unconsumed and offers the same bytes on the next poll, so congestion propagates back to the client's publication. A dead cluster session is not retryable and rejects with `GATEWAY_UNAVAILABLE`.
>
> The **outbound** leg cannot do this — egress must keep being drained or the session dies — so it drops and counts. A rising `droppedToClient` means a subscriber is gone or too slow; the fix is a larger term buffer or a faster subscriber, not a busier gateway.

Rejection reasons

Reason	Raised by	Meaning
`UNKNOWN_SECURITY`	gateway, engine	`securityId` is not on this shard
`MARKET_CLOSED`	engine	The security is `CLOSED`. A volatility halt looks like this
`PRICE_OUT_OF_BOUNDS`	engine	Outside the static collar around `staticReference`
`PRICE_OUT_OF_LADDER`	engine	Outside the allocated ladder. Should be unreachable: the ladder is configured strictly wider than the band
`BOOK_CAPACITY`	engine	The order pool's high-water mark for this book
`ORDER_EXPIRED`	engine	`expireDate` is before the current trading date
`UNKNOWN_ORDER`	engine	A cancel for an order that is not resting
`SELF_MATCH_PREVENTED`	engine	The aggressor's remainder was cancelled by SMP
`VOLATILITY_HALT`	engine	The aggressor's remainder was cancelled by a dynamic collar breach
`GATEWAY_UNAVAILABLE`	gateway	The cluster session is closed
`UNAUTHORIZED_PARTICIPANT`	gateway, engine	At the gateway: the participant is not one this gateway may act for — or, for a new order, is only `cancelOnly` here (4.3), and it never reaches the cluster. At the engine: a cancel whose participant or `origClOrdId` does not match the order it names
`INVALID_CL_ORD_ID`	gateway	A `clOrdId` or `origClOrdId` that is empty, longer than 20 characters, has a character outside the set, or has anything after its padding. The report echoes the 20 bytes as sent. Never reaches the cluster

Self-match prevention

The effective SMP id is `smpId` if non-zero, otherwise `participantId`; it is resolved once at order entry and stored on the order. The **aggressor's** strategy governs.

Strategy	Behaviour
`CANCEL_AGGRESSOR` (default)	Stop matching, cancel the aggressor's remainder, leave the resting order
`CANCEL_RESTING`	Cancel the resting order and continue matching

A self-match never prints a trade, so it never moves either reference price.

SMP applies in the opening auction too, which makes the uncross a **fixed-point loop**: cancelling removes volume, which moves the uncross price, which crosses a different set of orders. An auction has no aggressor, so the later-arriving order of a self-matching pair plays that role and its strategy governs. The loop terminates because each iteration cancels at least one order from a finite book; `engine.auction.maxPasses` is a safety valve, not part of the algorithm.

The auction

The uncross runs on `OPEN_AUCTION → CONTINUOUS` and selects a price by, in order: maximum executable volume, then minimum imbalance, then the surplus side, then the price nearest the reference.

> **THE AUCTION IS UNCOLLARED**
> Neither collar constrains the uncross price. That is what lets a halted security reprice and reopen. It is not unbounded in practice, since every participating order passed the static collar when it was accepted.

A volatility halt

The dynamic collar is measured against a **snapshot** of `dynamicReference` taken when the aggressing order arrived — never the live value, which that order's own fills are advancing. On breach:

- the breaching fill is suppressed, and prior fills stand;
- the aggressor's remainder is cancelled with `VOLATILITY_HALT` and is not booked;
- the security moves to `CLOSED` with its resting book intact;
- a `VolatilityHalted` event is emitted on L3, and nowhere else.

There is no automatic recovery and no `VOLATILITY_HALT` phase. Recovery is 6.4.

After a failover: no order is left unknown

When the leader fails, orders in flight end up in one of two states. Most never reached the log, so nothing happened to them. A few were sequenced, but their execution reports were lost while the new leader took over. Measured on one machine at 2,000 orders/s, about 65–95 per failover. Those orders rest or fill, and the client must be told.

Every execution report carries the participant's `reportSeq`: from 1, without gaps, across all of that participant's orders and every gateway. A client recovers like this:

1. **On a gap in** `reportSeq`, send a `ReportResendRequest` for that participant **bounded to the gap**: `fromSeq` the first `reportSeq` it has not received, `toSeq` the last before the next one it has. **After switching to another gateway**, or when an order has gone unanswered for longer than a failover takes, send one per participant **unbounded** (`toSeq` 0): what is missing may lie above everything received.
2. The engine replays the retained reports in that range, each with its original `reportSeq`. Count one you already have only once. Then it sends `ReportResendComplete`, with `nextSeq`.
3. **Once you hold every report below** `nextSeq`**, any order you sent before the request that still has no report was never sequenced.** Reject it to your own client, or send it again; either way it executes at most once. Still missing one (a report the gateway dropped on the way, above a bounded request's `toSeq`)? Settle nothing yet, and ask for that one first.

An unbounded request costs a replay of everything after the first missing report. Under load that is a lot: up to 2.4M duplicates were measured at the throughput ceiling, sent into an egress that was already full, so bound every request that can be bounded.

A completion with status `TRUNCATED` means the reports below `oldestRetainedSeq` are no longer retained. `engine.reportRetention` was smaller than the failover's traffic, or the shard was restarted since. Then:

4. **Send an order mass status** for that participant. It states every open order (original, cumulative and remaining quantity) and ends with `nextSeq`. Resume the participant's report sequence from there; the status stands in for the reports that were lost.
5. An order you sent that is neither answered nor open finished in the lost window, or never reached the log, and nothing can tell which. At most `oldestRetainedSeq - fromSeq` of them were sequenced. For fills, consult your trade records.

`most load` implements all of this, and `e2e/run-failover.sh` requires that no order is left unknown.

Order mass status

The state of every open order of one participant, as the engine holds it now: the reconciliation to run at the open, and after a truncated resend.

```
most status --participant 7 --shard 0
```

```
status: participant 7, every book on shard 0
  ORDER_STATUS  AAPL  orderId=1 clOrdId=1001  leaves 6 cum 4 of 10
status: COMPLETE -- 1 open orders, as of reportSeq 2 (next report is 3)
```

`--symbol SYM` narrows it to one security. It goes through the participant's own gateway, which checks it as it checks a cancel, and **it is answered**, unlike the operator commands of 5.3. A status report is not an event: it carries no `reportSeq` and is never replayed. The request costs one walk of the shard's resting orders, as a cancel-all does.

5.5 Market data operations

Four feeds, on four channels, plus the recovery feed. A subscriber's responsibilities are fixed by the flow-control choice (3.7): detect gaps by sequence number, and re-synchronise from the snapshot feed.

> **FEED SEQUENCE**
> Every book event carries a `seqNum` and a `shardId`. The sequence is stamped by the engine where the event is generated, so it is deterministic across nodes and survives failover, and it is snapshotted alongside the next order id. **Sequences are namespaced by shard** — each shard numbers from 1 independently — so several shards can share one multicast group and a subscriber tracks a sequence per shard.

Consumers should use `DepthFeedAssembler` and `FeedSequenceTracker` from the `client` module. The operator CLI, the control plane's console and any FIX market data adapter all use the same two, so a book on one screen cannot disagree with a book on another.

> **A SYNCHRONISED SUBSCRIBER IGNORES SNAPSHOTS, WITH ONE EXCEPTION**
> Installing an image over an already-synchronised book would **rewind** it, because increments past that sequence were applied directly and never buffered. The exception is an image whose sequence is at or ahead of everything applied, which already contains all of it — without that exception, a subscriber handed a stale image on join discards every later one and can only be rescued by an increment happening to arrive. On a quiet book, none does.

An empty book still sends a bracketed zero-level cycle, because “no liquidity” and “I cannot yet know” are different answers.

5.6 Snapshots and restarts

> **NOTHING TAKES A SNAPSHOT UNLESS SOMETHING ASKS**
>
> Without one, a restart replays the log from genesis — which is precisely what the snapshot exists to avoid. Four things ask:

`most cluster snapshot --ingress 0=HOST:PORT --identity ID --secret-file F`	Through consensus, so **every member** snapshots at the same log position, and the command prints the cluster’s **answer** — `snapshot taken, the cluster answered OK`, or the refusal. On a node with a registry the identity must be an `operator=true` entry (4.3). This is the one to use in production.
`most cluster snapshot --dir DIR`	The local control toggle, one member only
`most cluster shutdown`	Snapshot, then stop. The cluster host and the engine both exit 0 by themselves afterwards. Plain SIGTERM takes no snapshot
The control plane’s scheduler	At each session close, as the control plane’s operator identity (4.9)

A snapshot through consensus is only as real as the answer. Aeron’s own default authorisation grants **no** snapshot request, and until the registry authorised operators every one asked for over the network was refused — while being reported as requested, because only the offer was checked. The cluster’s recording log is the ground truth when in doubt:

```
java --add-opens java.base/jdk.internal.misc=ALL-UNNAMED \
  -cp "/opt/most/tools/lib/*" io.aeron.cluster.ClusterTool <cluster-dir> recording-log \
  | grep -o 'type=SNAPSHOT' | wc -l        # two entries per snapshot: consensus module and service
```

The `--add-opens` is not optional: without it ClusterTool fails at once with an `IllegalAccessError`, like every JVM running Agrona 2.x (2.1). In the dev stack the cluster directory is `/cluster/cluster` in the `cluster-host` container.

Leave the scheduler enabled in production: a daily snapshot at the close is what bounds restart time.

Restarting a node

```
most cluster shutdown --dir <the node's --dir>      # snapshot; both processes then exit 0 by themselves
systemctl stop most-engine most-cluster-host        # already stopped after a shutdown; needed after a close snapshot
sleep 15                                            # mark-file liveness is ~10s; 15 for margin
systemctl start most-cluster-host most-engine
journalctl -u most-engine | grep 'restored'
```

That last line is the check. The engine prints `restored N resting orders ... from a snapshot` specifically so that a real recovery is distinguishable from a node that found no snapshot and replayed its whole log, which renders the same book by a different route.

> **NEVER STOP THE ENGINE ON ITS OWN**
>
> If you stop the engine and leave the cluster host running, the consensus module sees the engine's Aeron client close, logs `Aeron client in service closed unexpectedly`, and terminates the cluster, which closes every gateway's session with it. An engine started afterwards is told `expected termination` and exits. Restart a node's processes together, cluster host first.

5.7 Measuring

`most load` drives a shard at a fixed rate and measures the round trip. The shard must already be defined and `CONTINUOUS`.

```
$ most load --symbol AAPL,MSFT --price-min 99.00 --price-max 103.00 \
      --rate 20000 --count 60000 --warmup 10000 --participant 7 --participants 2

load: sending 60,000 orders at 20,000/s (50 µs apart)
     1.0s  sent 20,001 (20,000/s)  reports 29,986  answered 19,941  in-flight 60  ack p99 7094.3
     2.0s  sent 40,001 (20,000/s)  reports 59,452  answered 39,902  in-flight 99  ack p99 6991.9
     3.0s  sent 60,000 (19,966/s)  reports 88,676  answered 60,000  in-flight  0  ack p99 6975.5

load: AAPL,MSFT  60,000 orders in 3.00s -- 20,000/s achieved (target 20,000/s)
  measured path  client → gateway → cluster consensus → engine → gateway → client
  band           99.00..103.00  qty 1..100  participants 7..8  warmup 10000
  offers         ok=60,000  backpressure-retries=0  dropped=0
  reports        88,676   new=30,546 trade=19,080 canceled=9,596 expired=0 rejected=29,454
  fills          528,026 qty traded, 7,695 orders filled on arrival (12.8%)
  unanswered     0 orders never saw a report
  REJECTED       29,454 x MARKET_CLOSED
  pacing         lateness n=50,000  p50=0.0 p90=0.0 p99=0.1 p99.9=22.3 max=1132.5 (µs)
  ack  response  n=50,000  p50=5275.6 p90=6537.2 p99=6975.5 p99.9=7622.7 max=8028.2 (µs)
  ack  service   n=50,000  p50=5275.6 p90=6537.2 p99=6975.5 p99.9=7622.7 max=8028.2 (µs)
  fill service   n=7,695   p50=5238.8 p90=6512.6 p99=6983.7 p99.9=7721.0 max=7913.5 (µs)
```

Read it in this order:

1. `pacing lateness` **first.** If it is not small, the generator was the bottleneck and the run says nothing about the exchange. Its schedule is absolute — `start + i × delay` — because a relative sleep drifts and then catches up in bursts.
2. **The reject counts.** The run above is a worked example of a run that must be discarded: 29,454 `MARKET_CLOSED` rejections mean one of the two securities halted partway through, so half the orders never reached a book. Two things silently invalidate a run in exactly this way — a `maxOrders` too small for the rate, which turns everything into `BOOK_CAPACITY`, and a price band outside the static collar or the ladder. That is why the summary prints them.
3. **Both latencies.** *Service time* is measured from the actual send; *response time* from the scheduled send. Quoting only the first is coordinated omission and hides exactly the queueing that appears at the rate you are trying to find.

Run the generator on a separate machine, never on a cluster node. Above ~1,500,000/s one `most load` process is at its own limit (its pacing lateness passes 1 ms), so a knee measured there is partly the generator's.

What a shard actually carries

`e2e/run-sweep.sh` drives a series of rates, validates each one before believing it, and prints a row block for the record. `SECURITIES=n` drives *n* securities, and `RATES` **is the aggregate across them** — `SECURITIES=10 RATES=1000000` is 100,000/s per security.

Measured, single node, ten securities, development machine with the desktop closed:

Driver threading and egress	Sustained (aggregate)	Per security
`SHARED` driver, UDP egress (the defaults)	~350,000 orders/s	~35,000/s
`DEDICATED` driver, UDP egress	~500,000–550,000 orders/s	~50,000–55,000/s
`DEDICATED` driver, **IPC egress**	**~1,500,000 orders/s**; 1,000,000/s held for 8 s at an 83 µs median, 177 µs p90	~150,000/s

The UDP rows are where the driver's sender runs out of datagrams for execution reports, and the engine waits behind it (4.8, "The egress channel"). They are not where the development laptop ran out of cores, as this section once said. A 16-core host with every thread on its own core measured the same limit lower, at ~275,000–300,000/s, because its cores are slower. **The IPC row is where the engine's own thread fills**: fully busy by ~2,100,000/s at 0.43 µs a new order, with the gateway close behind at 93%, and the laptop's cores and its one generator running out at the same time.

Plan capacity against the row for the configuration you actually run, not against the design target of 100,000/s per security across ten. The target is met only in the IPC row, which needs the gateway on the leader's media driver (4.8), and only on one node. A multi-node cluster replicates the log to followers over UDP through the same sender, and has not been measured. Two findings behind these numbers are worth carrying into any capacity conversation:

- **The ceiling is aggregate, not per security.** Ten books sustain the same *total* rate one book does; fan-out costs 8% of the work of a single order and buys no throughput. A shard is one thread's worth of shared path — one ingress, one consensus module, one archive, one log — and adding securities divides it rather than multiplying it.
- **With UDP egress, matching is not the constraint.** The engine's own whole-message p50 is 0.38–0.50 µs, about 23% of its thread at the UDP ceiling. The rest of that thread is spent waiting on a full egress publication (5.8). With IPC egress it is the constraint: the engine is the first thread to fill.

> **A RATE ABOVE THE KNEE IS A QUEUE, NOT A LATENCY**
>
> Past the sustainable rate the shard still accepts everything — no rejects, no drops, every order answered — and the delay becomes a backlog that never drains. The `achieved` figure on such a run is the rate the *generator offered*, not one the shard sustained, and its p50 is the queue draining. `run-sweep.sh` marks the row `SATURATED` and names the highest rate the shard kept up with. Never quote a `SATURATED` row's latency.

> **ATTRIBUTION**
>
> `e2e/run-attribution.sh` splits a round trip by stage and writes `.hgrm` histograms meant to be diffed across a change. On an idle development machine the gateway and engine own **0.8 µs of a 38 µs** round trip — 2.0%, falling to 0.9% as the rate rises — and the engine's whole-message p50 is 0.46 µs at one security, 0.50 µs at ten. The number to watch after a change is the *share*, not the round trip: real network hops add to the denominator and dedicated cores subtract from it. It takes the same `SECURITIES` and `CLUSTER_HOST` knobs as the sweep.

5.8 Monitoring

What to watch

Signal	Where	Meaning
`droppedToClient`	gateway	Undeliverable execution reports. The outbound leg drops and counts by design; a rising rate means a subscriber is gone
`clusterBackpressure`	gateway	Transient and retried, so not a loss — but a sustained rate means ingress is saturated
`untrackedReports`	gateway	Reports the engine could not state an `origQty` for. Only orders restored from an older snapshot should produce these
`invalidClOrdIdRejects`	gateway	Orders and cancels refused `INVALID_CL_ORD_ID`. Non-zero means an adapter is passing through ids the exchange cannot carry
`refusedSchemaVersion`	gateway	Orders and cancels from a client, or reports from an engine, on a schema version before 8, dropped unanswered. Non-zero means something was not upgraded (6.7)
`gaps`, `eventsMissed`	market-data	Feed sequence gaps. Occasional under `MaxMulticastFlowControl`; sustained means a subscriber cannot keep up
`foreignShard`	market-data	Book events for another shard. Means this process is pointed at the wrong engine
`imagesApplied`	market-data	A restarted process expects this to be non-zero. Zero with a book that stayed empty means the engine never sent one
`rejectedDefinitions`	engine	A `SecurityDefinition` was refused. Commands are unacknowledged, so this counter is the only signal
`registryReloadFailures`	engine, cluster-host, gateway	A registry that could not be read or was for another shard. The one in force still applies, so this is a quiet wrong rather than an outage
`rejectedSessions`	cluster-host, at shutdown	Sessions refused at connect: wrong secret, unknown gateway, or no credentials on a node with a registry
`unauthorizedRejects`	gateway	Orders and cancels refused `UNAUTHORIZED_PARTICIPANT`. Non-zero means someone reached this gateway as a participant it does not serve
`refusedCommands`	gateway	Operator commands consumed unsent because this gateway is not an operator. Non-zero with markets not moving means the control plane or CLI is pointed at the wrong gateway (4.9)
`undeclaredParticipantMessages`	engine	Orders and cancels from a gateway that does not list the participant. Counted, never refused; non-zero means a gateway is not enforcing the registry the engine holds
Leader changes	consensus	Any unexplained one is worth a look

> ⚠ **THERE IS NO HEALTH OR METRICS ENDPOINT ON THE CORE PROCESSES**
>
> Everything above is a log line or a counter printed at shutdown. A scrape endpoint per process is what would make this mechanisable.

The exchange's own view of itself

`GET /api/status` is what the exchange is actually doing, as opposed to what the database says it should be. It is derived from L3 and from the discovery broadcast.

```
{
  "connected": true,
  "directory": { "version": "9181280125937456696", "securities": 2, "shards": [0],
                 "lastSeenAt": "2026-09-26T14:02:07.570Z", "incompleteBroadcasts": 0 },
  "securities": [
    { "securityId": 1, "shardId": 0, "phase": "CLOSED",
      "halt": { "at": "2026-09-26T14:02:05.698Z", "collarReference": 10005000000,
                "attemptedPrice": 12500000000, "breachedBound": 2001000000,
                "aggressorSide": "BUY", "clearedAt": null },
      "lastTradePrice": 10050000000, "lastTradeQty": 60 },
    { "securityId": 2, "shardId": 0, "phase": "CONTINUOUS", "halt": null,
      "lastTradePrice": 9996000000, "lastTradeQty": 36 }
  ],
  "feedGaps": 0, "eventsMissed": 0, "eventsSeen": 35528,
  "routingDrift": []
}
```

Captured from the development stack (fields trimmed for width) after a 20,000-order load run across both securities and a buy swept AAPL towards 125.00 against a reference of 100.05: the dynamic collar (±20%) halted it at the breaching level, so security 1 is `CLOSED` with a `halt` block whose `clearedAt` stays null until an operator reopens it (6.4). **35,528 book events and not one gap** is what a healthy feed looks like. An older capture read `"feedGaps": 9543` against `"eventsSeen": 9552` — not a lossy feed but a defect in the control plane, which counted a sequence only for the four book events it interprets while the engine numbers all seven, so every order event it ignored read as a gap and real loss was invisible.

The three feed counters, once they mean what they say:

Field	Healthy	What a non-zero value means
`eventsSeen`	climbing	Book events the control plane has read off L3
`feedGaps`	0, or rare	Jumps forward in the sequence: the subscriber missed messages. Occasional under `MaxMulticastFlowControl` is by design; sustained is a real problem, and the fix is not changing flow control (6.6)
`eventsMissed`	0	How many messages those gaps accounted for

`routingDrift` reports where the database and what discovery is actually broadcasting disagree. Commands route from the **database**, not the directory — the control plane is the authority on topology and a command must still be sendable when discovery is down — so where the two disagree, that is said out loud rather than one silently winning.

Reading Aeron's own counters

Every Aeron component — the media driver, the archive and each cluster component — publishes counters into the driver's `cnc.dat`. `most counters` reads them **with no Aeron client**, so it is safe to point at a shard under load and cannot perturb what it is measuring.

```
most counters --aeron-dir /dev/shm/aeron-most           # a snapshot of everything non-zero
most counters --aeron-dir … --interval-ms 4000 --samples 3   # rates of change, busiest first
most counters --aeron-dir … --all --match 'luster|archive' # including the still ones, filtered
```

`--interval-ms` is the mode that finds things. A counter's value is rarely interesting; its slope usually is, and the shape to look for is **a position counter that stops advancing while the one feeding it does not**. Reach for this before theorising about where time goes — it is what identified the driver threading mode, and then the egress channel, as the shard's ceilings (4.8).

> **A CLEAN COUNTER SHEET DOES NOT MEAN NOTHING IS SATURATED**
>
> Aeron reports queues, positions, duty-cycle breaches and errors. A stage that is simply **full** breaches none of them, so "every counter scaled with the offered rate and nothing exceeded a threshold" narrows an answer to *not a buffer, a window, a disk or a stall* and no further. Nor can `top` finish the job: the engine, gateway and market-data busy-spin and read ~100% of a core whether working or idling (3.4). The next instrument after the counters is the duty cycle, below.

> **A PUBLICATION AT ITS LIMIT**
>
> A publication's `pub-lmt` minus its `pub-pos` is how far its writer may still go. At zero the writer waits, and a writer that retries inside its work, as the engine does, waits **busy**. The engine's execution reports to the gateway are cluster egress, stream `102`. If its headroom is at zero and the driver's `snd-pos` for the same stream is megabytes behind `pub-pos`, while the receiving side has caught up, then the driver's sender is the stage that's full and the engine only looks full. That is the UDP ceiling of 4.8. `snd-bpe` doesn't show this: it counts the sender meeting the *receiver's* window. Sample the positions with `most counters --aeron-dir … --match 'pub-pos|pub-lmt|snd-pos|sub-pos' --all --interval-ms 3000`, or with `e2e/run-attribution.sh`'s `COUNTERS_MATCH`.

> **A COUNTER THAT MOVES NON-LINEARLY NAMES A PLACE TO LOOK, NOT A CAUSE**
>
> Sender flow-control back-pressure on the cluster ingress channel rose 89-fold at the knee, which looked conclusive and pointed at the ingress term length. Raising it 256-fold moved the sustainable rate not at all — back-pressure on a channel is what a slow *consumer* looks like from the publisher's side, so a closing window is as likely to be the symptom as the cause. Change it and re-measure before believing it.

How full each thread is

Every loop on the order path can publish its **duty cycle** — the share of wall time it spends in iterations that found work — as an Aeron counter labelled `duty-ns: <thread>`. It is the one reading that tells a busy thread from a spinning one. Turn it on with each process's metrics switch (`engine.metrics`, `gateway.metrics`, `md.metrics`) and `most cluster --duty` for the driver, archive and consensus module; all are on in the development stack. Then:

```
$ most counters --aeron-dir build/attr-duty-check/aeron --match '^duty-ns' --all --interval-ms 3200
counters: build/attr-duty-check/aeron/cnc.dat  pid=55343

  sample 1 of 1 over 3.20s
     id                value         per second  label
    166           4706046275        918,693,319  duty-ns: driver sender   = 91.9% of a core
    167           3523624174        688,061,980  duty-ns: driver receiver   = 68.8% of a core
    168           1117142847        203,768,413  duty-ns: archive   = 20.4% of a core
    165           1006793108        178,289,139  duty-ns: driver conductor   = 17.8% of a core
    170            899559798        161,337,009  duty-ns: gateway anonymous   = 16.1% of a core
    169            780158699        134,305,447  duty-ns: engine service   = 13.4% of a core
     94            705590536        119,773,332  duty-ns: consensus-module   = 12.0% of a core
    171            362722381         65,071,360  duty-ns: market-data   = 6.5% of a core

  A duty-ns counter's rate is busy nanoseconds per second: the share of one core its
  thread spent working, which CPU% cannot show for a thread that spins while idle.
  A position counter that stops advancing while its feeder keeps going is the
  saturating stage. Sorted by absolute change, so the busiest is at the top.
```

That is `e2e/run-attribution.sh` sampling mid-load: one node at 250,000 orders/s across ten securities, `DRIVER_THREADING=DEDICATED`, with a gateway that connected without an identity — hence `gateway anonymous`; a registered one is labelled with its id, `gateway gw-0`. The duty cycle costs two clock reads per loop iteration and changes nothing about how a thread idles, so it can stay on.

Read it like this:

- **A thread that reaches ~100% is either full or waiting busy.** The engine's publication retries inside its work, so behind a full egress publication it reads 99.9% while doing nothing useful. The step at the development machine's UDP knee is exactly that: 20% at 500,000/s, 99.9% one step later. Read the egress headroom (above) before calling the engine full.
- **Do not judge the driver's sender or receiver by their duty cycle.** They read 90–98% at every rate measured, including runs where the kernel charges the sender ~2% of a core and runs where it carries a quarter of the traffic. So the reading says nothing about them yet; that is an open issue. Judge the sender by its positions instead: `snd-pos` against `pub-pos` (above).
- **A process spinning at a low duty is wasting a core.** On a machine without a core per thread that core is taken from somebody who needed it; set that process to `backoff` (3.4).

> **ON A MACHINE WITH FEWER CORES THAN BUSY THREADS, CORES DECIDE WHERE THINGS BREAK**
>
> The development laptop's ~550,000/s knee was first read as its core count: busy threads outnumbering its 10 performance cores. It is the egress limit of 4.8. Setting the gateway and market data to `backoff` moved it up a step, most plausibly by giving the driver's sender a core. The core count does bind higher up: with IPC egress, above ~1,500,000/s the shard's busy threads outnumber the laptop's performance cores, and a knee measured there describes the machine as much as the shard.

Who asked

`operator_audit` records every market-moving command against the operator who issued it, and records `sent`, **not** `applied`. A command the control plane could not deliver is recorded too, with `sent: false`; that is exactly as interesting to an investigation as one that went out.

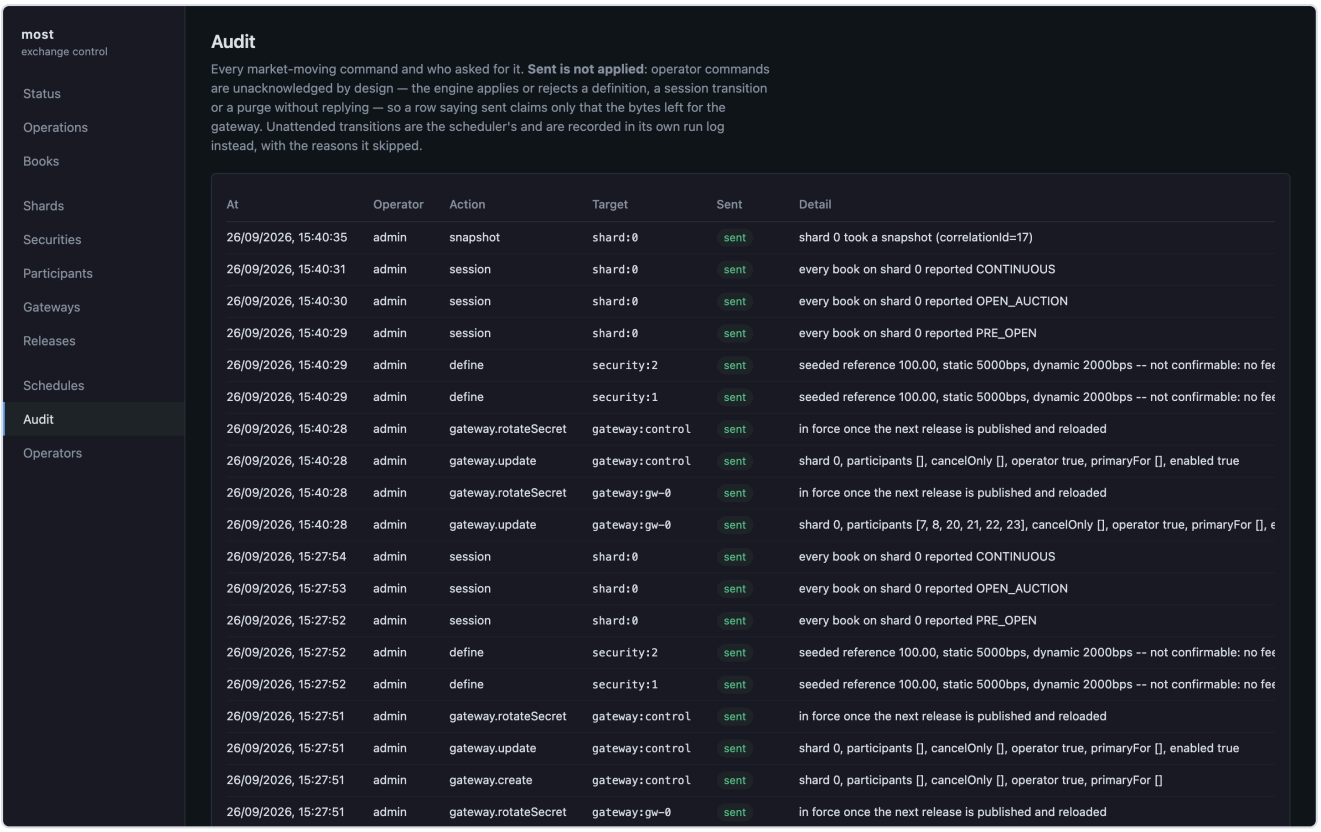


Figure 5.4 Audit. Every market-moving command, the operator who asked for it, and whether the bytes were sent.

Unattended transitions stay in the scheduler's own richer `schedule_run` log, which also records what it deliberately did *not* do.

Operator accounts

There is one role, `ADMIN`, with full access; the audit is what distinguishes who did what. Two ways in, for two different callers: a session cookie plus a CSRF token for the browser, and HTTP Basic — exempt from CSRF, because a request carrying its own credentials is not a forgery risk — for scripts.

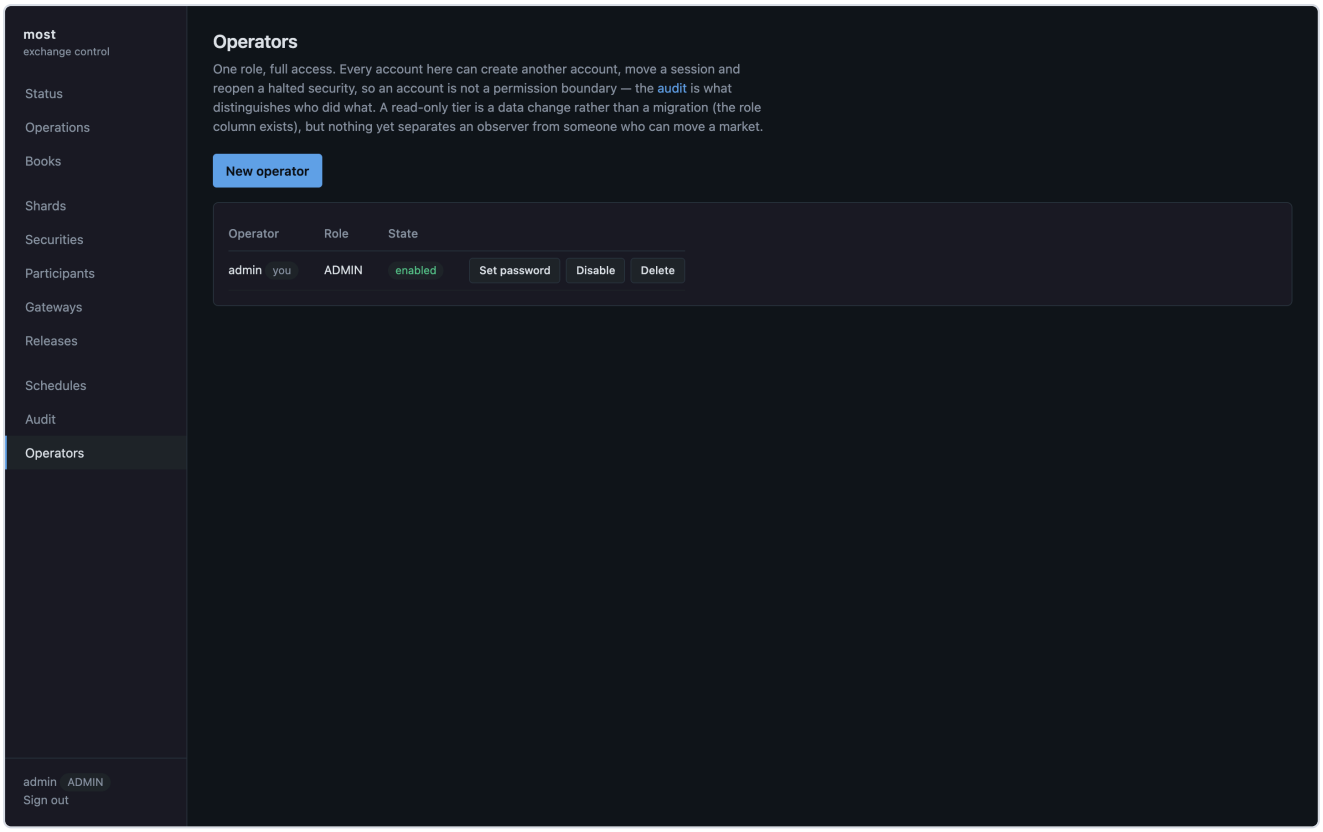


Figure 5.5 Operators. Accounts and their state. The column exists so that adding a read-only tier later is a data change rather than a migration.

6. Troubleshooting and maintenance

6.1 First moves

Four questions answer most incidents, in this order.

1. **Do all processes agree on geometry?** Grep every startup log for `fingerprint`. One distinct shard value and one distinct registry value, or stop here.
2. **What phase does the exchange think it is in?** `GET /api/status`, which reads L3. Not the database, and not the calendar.
3. **Is the directory being broadcast?** `most securities`, or `directory.lastSeenAt` in the status response. A `most` command that hangs is almost always waiting for it.
4. **What do the processes' own counters say?** They are printed at shutdown, so a process that was `SIGKILL`ed tells you nothing. That is one of the reasons SIGTERM matters (3.8).
5. **What does Aeron say, live?** `most counters --interval-ms 4000 --samples 3` reads the driver's, the archive's and every cluster component's counters without attaching a client, so it works on a shard that is currently misbehaving (5.8). Read the slopes, not the values.

6.2 Startup failures

Symptom	Cause and remedy
`missing required configuration key: engine.securitiesFile`	No default exists. Give the process its properties file, or set the key as a system property
`shard security file not found: ...`	The `current` symlink points nowhere, or the release was never fetched to this machine
`invalid ISIN 'US0378331006' for security 1`	Check-digit validation at boot. The control plane refuses the same value with the same sentence
`duplicate symbol in shard: [AAPL]`	Two securities in one shard file share a symbol or an id
`engine.participantRegistry is for shard 1, but this node serves shard 0`	A registry from another shard's release. Caught here rather than at the first misrouted report
`gateway id 'gw-x' is not in the participant registry`	The gateway's id and the registry disagree. Stops the process rather than becoming a cluster connection the consensus module refuses with no explanation
`gateway.gatewayId and one of ... must be set together`	An id with no secret cannot authenticate; a secret with no id has nothing to authenticate as
`no Aeron media driver found`	The driver is not running, or `*.aeronDir` points somewhere else
`could not reach the cluster`	The consensus module is not running, or `gateway.ingressEndpoints` names the wrong hosts
`Active media driver detected`	A previous driver's liveness timeout has not expired, or the Aeron directory was preserved across a restart. It must not be — it is memory-mapped buffers and a `cnc.dat`
`active mark file detected`	A node restarted within roughly ten seconds of the previous one stopping. Wait

Symptom	Cause and remedy
`matching-engine: refused to go on -- the log carries a version 7 NewOrderSingle`	The log since the last snapshot was written by an older version, or a gateway was not upgraded. Nothing was applied from it. Restart the old version, `most cluster shutdown`, then start the new one (6.7, “Upgrading to schema version 8”)
`could not create archive directory`	A volume or directory landed root-owned
`channel error - Cannot assign requested address`	A channel names a host whose media driver does not own that address. In the dev stack that means anything other than `shard0` (2.9)

6.3 The engine refuses to restore its snapshot

This is the system working. `loadSnapshot` reconciles the snapshot against the security file the node just booted from, and refuses rather than losing state.

```
matching-engine: refused to restore its snapshot -- security 2 (MSFT) is not in this shard's security file but holds 4 resting orders
```

Refusal	What to do
A security is gone from the file but holds resting orders	Restore the previous release, drain the book, then apply the change
`priceFloor`, `tickSize`, `levelCount` or `maxOrders` changed	Same — geometry cannot change under a populated book
An order falls outside the new ladder	Same
Resting-order counts do not add up	Do not restart around this. It means the snapshot and the file describe different things

The two legitimate cases are quiet and are logged with one line each: an **empty** book leaving the shard, and a new security joining.

> **A REFUSED RESTORE EXITS THROUGH THE SHUTDOWN BARRIER, NOT A HALT**
>
> Halting would skip the service container’s close, which leaves the cluster mark file carrying a live timestamp — so the operator who fixes the file and restarts immediately would be met with `active mark file detected` for the next ten seconds instead of a working node.

> **A BAD ORDER DURING A RESTORE DOES NOT CRASH THE NODE**
>
> Aeron’s `Image.poll` swallows an exception from its fragment handler, hands it to the client error handler, and advances the position anyway. The book would come back quietly wrong. This is why the restore range-checks every order itself and reconciles resting-order counts as a backstop — and why you should not assume a throw on this path would be loud.

6.4 Recovering a halted security

A halt sets **one security** to `CLOSED` and leaves the resting book intact. There is no halt phase, and `VolatilityHalted` is visible on L3 and nowhere else — a depth subscriber cannot tell a halt from a scheduled close.

Confirm it first:

```
"securities": [ { "securityId": 1, "phase": "CLOSED",
  "halt": { "at": "...", "collarReference": 10064000000, "attemptedPrice": 10167000000,
            "breachedBound": 100640000, "aggressorSide": "BUY", "clearedAt": null } } ]
```

Then reopen, in one operation:

```
curl -u admin:... -X POST localhost:8080/api/shards/0/reopen \
  -H 'Content-Type: application/json' -d '{"securityId":1,"referencePrice":10161000000}'
```

```
{ "shardId": 0, "securities": ["AAPL", "MSFT"], "succeeded": true,
  "steps": [
    { "command": "define AAPL", "sent": true, "confirmed": false,
      "detail": "seeded reference 101.61 -- not confirmable: no feed acknowledges a
                 SecurityDefinition, so a rejection would be silent" },
    { "command": "session PRE_OPEN shard 0",     "sent": true, "confirmed": true },
    { "command": "session OPEN_AUCTION shard 0", "sent": true, "confirmed": true },
    { "command": "session CONTINUOUS shard 0",   "sent": true, "confirmed": true } ],
  "warning": "a session transition is shard-wide: this moved AAPL, MSFT, not only AAPL" }
```

BOTH ORDERINGS ARE LOAD-BEARING

Re-seed the definition first. `staticReference` is reset only by an *executing* uncross, while orders are accepted from `PRE_OPEN` onward. If the halt moved price outside the old static band, the very orders needed to reopen are rejected by the stale collar before the auction that would have fixed it gets any — a deadlock that looks like nothing happening.

Walk the phases in full. The uncross runs only on `OPEN_AUCTION → CONTINUOUS`. Jumping straight to `CONTINUOUS` is accepted and silently skips the auction.

Doing it by hand is the same four steps:

```
most define  --symbol AAPL --reference 101.61 --static-collar 5000 --dynamic-collar 100
most session --phase pre-open     --shard 0
most session --phase open-auction --shard 0
most session --phase continuous   --shard 0
```

The scheduler will not do this for you, by design (5.2).

6.5 Order flow problems

Symptom	Diagnosis
A client's orders are all `MARKET_CLOSED`	The security is `CLOSED`. Check `/api/status` for a `halt` block — a halt and a scheduled close are indistinguishable from a depth feed
A client's orders are all `PRICE_OUT_OF_BOUNDS`	The static collar is anchored on a stale `staticReference`. Re-seed the definition
Everything is `BOOK_CAPACITY`	`maxOrders` is too small for the rate. Under a load test this invalidates the run entirely

Symptom	Diagnosis
A maker never receives its fills	The participant is not bound to a live session. Either no registry is configured — so routes are learned from traffic only, and a participant that has been quiet since connect is unreachable — or no connected gateway lists it
Orders rejected `UNAUTHORIZED_PARTICIPANT`	The gateway does not list that participant, or lists it only as `cancelOnly` (4.3). Check the gateway's startup or reload line, `gateway: identity=… participants=[…] cancelOnly=[…]`, against the registry in force
Session transitions, purges or definitions "sent" but nothing changes	They went through a gateway that is not an operator, which consumes them and counts `refusedCommands`. Operator commands are unacknowledged, so nothing else says so. Point the control plane at an operator gateway (4.9) or make that gateway one
A gateway will not connect: `session failed authentication` in its log, `presented no credentials` or `failed authentication as gateway` in the cluster host's	No identity configured on a node with a registry, or the wrong secret. Both are refused, never downgraded to anonymous
`most: no snapshot -- UNAUTHORISED_ACCESS`	The identity authenticated but is not an `operator=true` entry. Only an operator may request a snapshot through consensus
`most: no snapshot -- no answer from the cluster` or a connect timeout at `POLL_RESPONSE`	The egress endpoint is one the cluster cannot send to — `0.0.0.0` is the usual culprit. Name the host of the requester's media driver (4.9)
Duplicate orders	Two gateways subscribed to one `client.inbound.channel`. Give each its own (3.2)
Reports stop arriving but orders are accepted	`droppedToClient` is climbing: the outbound subscriber is gone or too slow
Orders are accepted but nothing matches	The phase is not `CONTINUOUS`. Booking without matching is correct in `PRE_OPEN` and `OPEN_AUCTION`
A crossed book that never uncrosses	`CONTINUOUS` was reached from somewhere other than `OPEN_AUCTION`, so the uncross never ran
Everything is slow, but nothing is rejected, dropped or unanswered	The shard is past its sustainable rate and you are measuring a backlog. Check the offered rate against 5.7, and check `--driver-threading` — on the default `SHARED` the ceiling is 1.6x lower than it needs to be (4.8). Then check the egress publication's headroom: at zero, with the driver's sender behind, the UDP egress channel is the limit and the engine only looks full (4.8, 5.8)
Latency degrades suddenly rather than gradually	Expected. The knee is abrupt: there is no gentle degradation to alert on before it becomes a stall, which is why the offered rate has to be watched rather than inferred from latency

> **AN ANONYMOUS GATEWAY CAN ONLY REACH A NODE WITHOUT A REGISTRY**
>
> A node with a registry refuses a session with no credentials, so the old quiet failure — an anonymous gateway trading perfectly well and losing only the fills of participants that had gone quiet — is now a refusal at connect. On a node **without** a registry it still happens, and the gateway enforces nothing there either.

6.6 Market data problems

Symptom	Diagnosis
`feedGaps` and `eventsSeen` in `/api/status` track each other	A control plane older than the feed-gap fix, counting every event it did not interpret as a gap. Upgrade; a healthy shard reports zero or near it (5.8)
A subscriber shows nothing and says it is waiting	It has not received a snapshot yet. A full pass is `md.snapshot.cycleMs × securities`
A subscriber shows nothing after the engine restarted	It restarted too, and has no book. Request a book image (5.3)
`gaps` climbing steadily	A subscriber cannot keep up. Expected occasionally under `MaxMulticastFlowControl`; sustained means a real problem, and the fix is not changing flow control
The feed stops minutes after everything looked fine	No IGMP querier on the VLAN, so snooping switches aged out the group
`foreignShard` non-zero	This market-data process is subscribed to another shard's engine
A halt is invisible	It is only on L3. Subscribe to L3, not L1 or L2
`imagesDiscarded` non-zero	Images arrived at an already-synchronised book that was ahead of them. Expected on join; sustained means something is republishing stale images

6.7 Maintenance procedures

Changing geometry

Geometry changes are applied by restarting, and the snapshot reconciliation (6.3) is what makes that safe.

```
1. Author the change in the control plane and publish a release.
2. Drain, or confirm empty, every book whose geometry changed or which is leaving the shard.
3. Fetch the release to every machine and move `current`.
4. Restart the cluster host and engine on each node in turn, waiting out the mark-file timeout.
5. Restart gateway, market-data and discovery so they read the new file too.
6. Compare fingerprints.
```

Changing who speaks for whom

A participant list change does **not** restart a node.

A participant list change restarts **nothing** — not a node, and not a gateway.

```
# 1. Publish from the control plane, then point `current` at it on every machine.
ansible shard0,gateways -a "ln -sfn /etc/most/releases/000042 /etc/most/current"

# 2. Watch the node processes adopt it.
journalctl -u most-engine -f | grep -m1 'registry: reloaded'

# 3. And each gateway, which prints its own new grants -- the line that says it is in force.
journalctl -u most-gateway -f | grep -m1 'gateway: now'
```

Check `registryReloadFailures` rather than assuming a silent success — a bad file is ignored, not fatal (4.3). `engine.participantRegistry.reloadMs=0`, `gateway.participantRegistry.reloadMs=0` and `--participants-reload-ms 0` turn the poll off if a deployment would rather restart.

Revoking a participant is two releases and, if it does not withdraw its own orders, one command. First move it from `participants` to `cancelOnly` on each gateway that lists it and publish: new orders are refused at once, and it can still cancel what is resting. To withdraw them for it, run `most cancel-all --participant N --shard S` (or the Operations page's *Cancel a participant's orders*) **after** the release is in force, on every shard it trades on (5.3). Once its orders are gone, remove it.

Replacing or adding a gateway

Gateways hold no state, so this is ordinary:

1. Create the gateway in the control plane and record the secret it returns **once**.
2. Publish a release, and deploy the registry to the cluster nodes.
3. Give the new gateway its own client inbound and outbound endpoints — never a peer's.
4. Start it, and point its adapters at it.

Removing one is the reverse; its adapters reconnect elsewhere and its in-flight orders are reported correctly by whichever gateway the client next uses.

Rotating a gateway secret

`PUT /api/gateways/{id}/secret` issues a new one and returns the plaintext once. Publish, deploy the registry, and restart that gateway. The nodes pick the new digest up without restarting, so the window in which the old secret still works is one poll interval.

Upgrading a node

Rolling, one member at a time, waiting for the group to be healthy between each:

```
most cluster snapshot --ingress 0=shard0-a:20110 \
     --identity operator --secret-file /etc/most/operator.secret   # every member, same position
systemctl stop most-engine most-cluster-host        # on one node only
# deploy the new binaries
sleep 15
systemctl start most-cluster-host most-engine
journalctl -u most-engine | grep 'restored'
```

> **A WIRE CHANGE IS NOT A ROLLING UPGRADE**
>
> Nodes running different schema versions over one log would diverge. Since version 8 the schema version is part of the engine fingerprint, so they refuse instead (4.2). Treat any change to `message-schema.xml` as requiring a coordinated restart of the whole shard, and re-run `e2e/run-e2e.sh` first — it exists because unit tests cannot see the wire.

Upgrading to schema version 8

Version 8 changed the layout of orders, cancels and execution reports in place (5.4), so the log written by an older version cannot be read by version 8. The upgrade is therefore a whole shard at once, through a snapshot at the end of the log:

```
most cluster shutdown --dir <the node's --dir>      # on version 7: snapshot, then the cluster
stops
systemctl stop most-engine most-cluster-host        # version 7's processes stay up after it;
version 8's exit
# install version 8: the cluster host, the engine, the gateways, market data and discovery together
sleep 15
systemctl start most-cluster-host most-engine
journalctl -u most-engine | grep 'restored'         # restored N resting orders ... from a snapshot
```

- **Open orders survive.** A snapshot from before version 8 restores, and each order's integer `clOrdId` becomes its decimal string: order `1001` is `"1001"` from then on, and a cancel names it so.
- **Without a snapshot at the end of the log, version 8 refuses** with `refused to go on -- the log carries a version 7 NewOrderSingle` and exits non-zero. Nothing is lost: start the old version, run `most cluster shutdown`, and upgrade again.
- **Upgrade every gateway and every adapter with the shard.** A gateway on version 8 drops an older client's orders unanswered, since it can send nothing that client could read, and counts `refusedSchemaVersion`. Adapters need the version 8 SDK.

On the dev stack the same steps are Compose commands. This is the dev stack's own upgrade, with 4418 orders resting:

```
$ docker compose exec -T cluster-host most cluster shutdown --dir /cluster
Member [0]: SHUTDOWN applied successfully
cluster: shutdown requested (a snapshot is taken first) via /cluster/cluster

$ docker compose stop engine gateway market-data discovery cluster-host   # version 7 stays up
$ docker compose --profile cli build                                      # version 8 images
$ docker compose up -d cluster-host engine gateway market-data discovery
$ docker compose logs engine | grep -E 'engineFingerprint|restored'

matching-engine: shard=0 fingerprint=3e04cf2b9902f08a engineFingerprint=c6befe8f7c5640c6 ...
matching-engine: restored 4418 resting orders across 2 books from a snapshot,
                 nextExchangeOrderId=20032 nextBookEventSeqNum=63 reportSequences=2
```

The shard `fingerprint` is unchanged, because the securities are. The `engineFingerprint` is not (it was `47e0ef48f9a52f14` on version 7), because the schema version is part of it. From version 8 on, `most cluster shutdown` stops the `cluster-host` and `engine` containers itself, both with exit code 0; `docker compose ps -a` shows them `Exited (0)`.

`e2e/run-upgrade.sh` runs exactly this, on one node: orders resting on version 7, the shutdown, the restore on version 8, a cancel by an old id, and the refusal of a version 7 log tail.

> **NOT YET RUN ON THREE MEMBERS**
> On one node the procedure is checked end to end. On a three-member shard, `most cluster shutdown` is expected to snapshot and stop every member at the same log position; confirm that every member's processes have exited before installing version 8.

Checking a native image you cannot run

The native build needs `--add-exports=java.base/jdk.internal.misc=ALL-UNNAMED`. A missing export does **not** fail the build: it silently omits the class, and the binary dies on its first `UnsafeBuffer` — and only once a real Aeron control file exists, so a smoke test against a *missing* driver passes and hides it.

```
grep -c 'jdk.internal.misc.Unsafe' /opt/most/bin/engine     # 0 = broken, 3 = correct
```

Starting the binary without a media driver does not check this: it exits on the expected driver timeout before the first buffer is ever wrapped.

6.8 Reading the logs

Line	Process	Means
`cluster: started, awaiting shutdown signal`	cluster-host	The consensus module is up. The engine may now attach
`matching-engine: started, awaiting shutdown signal`	engine	The service container is attached
`matching-engine: restored N resting orders ... from a snapshot`	engine	A **real** recovery, as distinct from a log replay
`gateway: identity=gw-0 registry=... participants=[7, 8] cancelOnly=[] operator=true`	gateway	What this gateway will **enforce**: who may place, who may only cancel, whether operator commands pass
`gateway: now participants=[...] cancelOnly=[...] operator=...`	gateway	A reloaded registry is in force for this gateway. The line to wait for after publishing a revocation
`gateway: no cluster identity`	gateway	Connecting anonymously and enforcing nothing. A node with a registry refuses it
`cluster: session N presented no credentials; ...`	cluster-host	An anonymous client refused by a node with a registry
`gateway: cluster session closed; orders can no longer be forwarded`	gateway	Terminal for this process
`cluster: registry: reloaded`	cluster-host, engine	A new participant registry came into force, with both fingerprints
`discovery: version=... shards=[0] securities=2`	discovery	The universe version now being broadcast

Shutdown lines carry the counters, which is why an orderly stop — SIGTERM or `most cluster shutdown` — matters:

```
gateway: stopped. forwardedToCluster=5020 clusterBackpressure=0 sentToClient=10971
         droppedToClient=0 standbyRejects=0 activations=1 rejectedLocally=0 unreachableRejects=0
         undeliverableCommands=0 unauthorizedRejects=0 refusedCommands=0
         invalidClOrdIdRejects=0 refusedSchemaVersion=0 untrackedReports=0

market-data: stopped. gaps=0 missed=0 foreignShard=0 droppedL1=0 droppedL2=0 droppedL3=0
             snapshots=929 droppedSnapshot=0 imagesApplied=1 imagesDiscarded=0 imageOutOfBand=0
```

6.9 Consolidated list of what is not implemented

Area	What is missing	Section
Cluster host	Member id is hardcoded to 0; no `--member-id`	3.6

Area	What is missing	Section
Capacity	With UDP egress a shard sustains ~350,000 orders/s aggregate on the default threading and ~500,000–550,000 with `DEDICATED`. The driver's UDP sender on egress sets that limit. With IPC egress it sustains ~1,500,000, and there the engine thread fills first. The design target of 100,000/s per security across ten is met only with IPC egress, only on one node, and IPC egress needs the gateway on the leader's media driver. Multi-node log replication over UDP is unmeasured	4.8, 5.7
Gateway placement	Nothing keeps IPC egress working across a failover: a gateway that follows the leader is neither designed nor tested	3.2, 4.8
Build	`engine.march` is x86-only	3.10
Discovery	One order-entry channel per shard, so several gateways cannot be advertised individually. Not a capacity lever below ~2,000,000/s: a second gateway raised no ceiling, and the one gateway reaches 93% only at ~2,100,000/s with IPC egress	3.2, 5.7
Market data	A feed is one channel; multicast and dynamic MDC cannot coexist	3.7
All processes	No health or metrics endpoint; monitoring is log lines and shutdown counters	5.8
Boot	Processes do not verify the release fingerprint they read against what was published	4.11
Engine	`--gc=epsilon` is proven by three measurements, now run by CI on every push, but is off by default pending a soak measured in hours	—
Production	None of section 3 has been run as described	3.10

7. Index

References are to section numbers. **Bold** marks where a term is defined.

7.1 Terms and concepts

A–C

Aeron directory 2.9, 4.8, 6.2
Anonymous session, refused 4.3, 4.8, 6.5, 6.8
Archive 3.3, 3.6, **4.8**, 5.6
Auction, opening 1.6, **5.4**
Auction, uncollared 5.4
Back-pressure, cluster egress 4.8, **5.8**
Back-pressure, cluster ingress 5.4, 5.8
Busy-spin idle strategy **3.4**, 4.4, 5.8
Cancel-only participant **4.3**, 6.7
Book capacity 4.2, 5.4, 6.5
Book event stream 1.2, 3.2, 4.4, 4.6
Book image **5.3**, 6.6
Circuit breaker, see Collar
Client order id (`clOrdId`, `origClOrdId`) **5.4**, 6.2, 6.7
Collar, dynamic **5.3**, 5.4
Collar, static 4.2, **5.3**, 5.4, 6.5
Conflation, depth images 2.8, 4.9
Consensus module 1.2, 3.6, 4.8
Coordinated omission 5.7
Core isolation **3.4**, 4.8
CSRF 2.8, 5.8
cumQty **2.6**, 4.5, 6.5

D–G

Determinism **1.1**, 4.4, 5.2
Directory, see Discovery
Discovery broadcast 1.2, 2.5, **4.7**, 5.8
Duty cycle 3.4, 4.4, 4.8, **5.8**
Dynamic MDC 2.9, 3.7
Egress, cluster 4.5, **4.8**, 5.4, 5.7, 5.8
Egress over IPC 3.2, 4.5, **4.8**, 5.7
Expiry, order **5.1**, 5.3
Feed sequence **5.5**, 6.6
Fingerprint, registry 2.4, 4.3
Fingerprint, shard geometry **1.4**, 2.4, 3.9, 4.11, 6.1
Fixed point, 8 decimals **1.5**, 4.2
Flow control, multicast **3.7**, 6.6
Gateway enforcement **4.3**, 5.4, 6.5
Gateway identity 4.3, 4.5, 6.7
Geometry, ladder **4.2**, 6.3, 6.7

H–O

Halt, volatility 1.6, **5.4**, 6.4
Huge pages **3.3**
Idle strategy 2.9, **3.4**, 4.4, 4.5, 4.6
IGMP querier 3.7, 6.6
Ingress, cluster 3.6, 4.5, 4.9
IPC egress, see Egress over IPC
ISIN validation 4.2, 6.2
L1 / L2 / L3 1.2, **4.6**, 5.5
Ladder, price **4.2**, 5.4
Leader-only publication 3.2
Mark file, liveness **2.12**, 3.8, 5.6, 6.2
Media driver 1.2, **2.3**, 2.9, 3.4
Member string **3.6**, 4.8
Metrics, hot path **4.4**, 4.5, 4.6, 5.7, 5.8
Multicast, market data **3.7**, 4.6
origQty **2.6**, 4.5, 5.8
Operator identity **4.3**, 4.9, 5.6

P–S

Participant binding **4.3**, 6.5, 6.7
Participant registry **4.3**, 4.4, 4.5, 4.8, 6.7
Primary gateway **4.3**, 4.10
Phase, session **1.6**, 5.1, 5.2
Purge, expired orders **5.1**, 5.3
Reject reasons **5.4**
Resend request, bounded to a gap **5.4**
Release, immutable **1.4**, 3.8, 4.11, 6.7
Reference price, static and dynamic **5.3**, 5.4, 6.4
Routing drift 5.8
Counters, Aeron **5.8**, 6.1
SBE **1.3**, 6.7
Schema version 8 **5.4**, 6.2, 6.7
Scheduler, session **5.2**, 3.9, 6.4
Self-match prevention **5.4**
sent vs confirmed **5.3**, 5.8
Shard **1.1**, 4.2
Shard security file **4.2**, 4.4–4.7
Snapshot, cluster **5.6**, 6.3, 6.7

Snapshot authorisation 4.8, 4.9, **5.6**, 6.5
Saturation, and the `SATURATED` marker **5.7**, 4.8, 6.5
Snapshot, depth recovery 4.6, **5.5**, 6.6
Sustainable rate **5.7**, 4.8
Synchronised, depth subscriber **2.7**, 5.5, 6.6

T–Z

Threading modes 3.4, **4.8**, 5.7
Throughput ceiling **4.8**, 5.7, 6.9
Tick size 1.5, 4.2
Topology, draft 4.10, 4.11
Trading date 5.1, 5.3
Uncross 1.6, **5.4**, 6.4
`UNAUTHORIZED_PARTICIPANT` **4.3**, 5.4, 6.5
Universe version 2.5, 5.8
Upgrade to schema version 8 **6.7**, 6.2
Zero allocation 2.2, 6.9

7.2 Commands

Operator CLI (`most`)

`most securities` 2.5
`most send` 2.6, 5.4
`most cancel` 2.6
`most book` 2.7
`most load` 5.7
`most define` 5.3, 6.4
`most session` 5.3, 6.4
`most purge` 5.3
`most image` 5.3, 6.6
`most cancel-all` 5.3, 6.7
`most cluster` 2.10, 4.8
`most counters` 5.8, 6.1
`most cluster snapshot` 5.6, 6.7
`--identity`, `--secret-file` 4.3, 5.6
`--order-entry-channel`, `--report-channel` 4.9
`most cluster shutdown` 5.6, 6.7

Control plane REST

`POST /api/auth/login` 5.8
`GET /api/status` 5.8, 6.1, 6.4
`GET /api/topology` 4.10
`POST /api/import` 4.11
`POST /api/releases` 4.11
`GET /api/releases/{v}/files/{name}` 4.11
`POST /api/gateways` 4.10, 6.7
`PUT /api/gateways/{id}/secret` 4.10, 6.7
`GET /api/gateways/registry/{shardId}` 4.10
`POST /api/securities/{id}/definition` 5.3
`POST /api/shards/{id}/session` 5.3
`POST /api/shards/{id}/purge` 5.3
`POST /api/shards/{id}/reopen` 6.4
`POST /api/shards/{id}/book-image` 5.3
`POST /api/shards/{id}/snapshot` 5.6
`GET /api/books`, `/books/stream` 2.8
`GET /api/books/status` 6.6
`PUT /api/schedules/{name}` 5.2
`PUT /api/shards/{id}/schedule` 5.2
`GET /api/scheduler/runs` 5.2
`POST /api/scheduler/tick` 5.2
`GET /api/audit` 5.8

Build and test

`./gradlew build` 2.2
`./gradlew installDist` 2.2, 2.3
`./gradlew :sbe:generateSbeCodecs` 1.3
`e2e/run-e2e.sh` 2.11, 6.7
`e2e/run-restart.sh` 2.11
`e2e/run-upgrade.sh` 2.11, 6.7
`e2e/run-attribution.sh` 2.11, 5.7
`e2e/run-epsilon-soak.sh` 2.11

7.3 Configuration keys and environment variables

Shard artifacts

`shard.id` 4.2, 4.3
`shard.securities` 4.2
`security.N.*` 4.2
`registry.gateways` 4.3
`gateway.<id>.secret` 4.3
`gateway.<id>.participants` 4.3
`gateway.<id>.cancelOnly` 4.3
`gateway.<id>.operator` 4.3
`participant.<id>.primary` 4.3

Engine

`engine.securitiesFile` 4.4, 6.2
`engine.participantRegistry` 4.4
`engine.participantRegistry.reloadMs` 4.4, 6.7
`engine.aeronDir` 4.4
`engine.clusterDir` 4.4
`engine.serviceId` 4.4
`engine.bookEvent.channel` / `.streamId` 4.4
`engine.auction.maxPasses` 4.4, 5.4
`engine.backpressure.alertThreshold` 4.4
`engine.metrics`, `.stages`, `.file` 4.4

Gateway

`gateway.securitiesFile` 4.5
`gateway.participantRegistry` 4.5
`gateway.participantRegistry.reloadMs` 4.5, 6.7
`gateway.gatewayId` 4.5, 6.2
`gateway.credentialToken` / `TokenFile` 4.5
`gateway.ingressChannel` 3.6, 4.5
`gateway.ingressEndpoints` 3.6, 4.5
`gateway.egressChannel` 4.5, 4.8
`gateway.client.inbound.*` 4.5, 3.2
`gateway.client.outbound.*` 4.5

`gateway.metrics`, `.file` 4.5

Market data and discovery

`md.securitiesFile` 4.6
`md.bookEvent.*` 4.6
`md.l1.*`, `md.l2.*`, `md.l3.*` 3.7, 4.6
`md.snapshot.*` 4.6, 6.6
`discovery.shards` 4.7
`discovery.shard.N.*` 4.7
`discovery.channel` / `.streamId` 4.7
`discovery.intervalMs` 4.7

Cluster host flags

`--dir`, `--aeron-dir` 4.8
`--archive-dir`, `--cluster-dir` 4.8
`--members`, `--host` 3.6, 4.8
`--participants` 4.8
`--participants-reload-ms` 4.8, 6.7
`--driver-threading` 3.4, **4.8**, 5.7
`--archive-threading` 3.4, **4.8**
`--ingress-term-length` 4.8
`--fresh` / `--keep` 4.8

Control plane

`CONTROL_DB_*` 4.9
`CONTROL_PORT` 4.9
`CONTROL_RELEASE_DIR` 4.9
`CONTROL_COOKIE_SECURE` 2.9, 4.9
`CONTROL_ADMIN_USER` / `_PASSWORD` **4.9**
`CONTROL_AERON_ENABLED` / `_DIR` 4.9
`CONTROL_DISCOVERY_*` 4.9
`CONTROL_L1_*` / `L2` / `L3` 4.9
`CONTROL_SNAPSHOT_*` 4.9
`CONTROL_DEPTH_PUBLISH_MS` / `_MAX_LEVELS` 4.9
`CONTROL_CLUSTER_INGRESS_<shardId>` **4.9**, 5.6
`CONTROL_CLUSTER_IDENTITY_<shardId>`, `…SECRETFILE_…` **4.9**, 5.6
`CONTROL_CLUSTER_OPERATORCHANNEL_<shardId>`, `…OPERATORSTREAM_…` **4.9**, 6.5
`CONTROL_CLUSTER_EGRESS_CHANNEL` **4.9**, 6.5
`CONTROL_SCHEDULER_ENABLED` / `_INTERVAL_MS` 4.9, 5.2

Aeron and JVM

`aeron.term.buffer.length` 3.5
`aeron.ipc.term.buffer.length` 3.5
`aeron.socket.so_sndbuf` / `so_rcvbuf` 3.5
`aeron.rcv.initial.window.length` 3.5
`--add-opens java.base/jdk.internal.misc` 2.1
`--add-opens java.base/sun.nio.ch` 2.1
`--add-exports java.base/jdk.internal.misc` 6.7
`--install-exit-handlers` 3.8
`engine.march` 3.10

7.4 Further reading

Document	Covers
`docs/Design.md`	The authoritative specification: determinism, the performance budget and memory footprint, the wire and the feeds, the native build
`docs/ControlPlane.md`	The control plane, its schema, releases, live control, authentication and scheduling
`docs/ProdDeployment.md`	The production tier in more detail than section 3
`docs/LocalTesting.md`	The manual walkthrough, and the benchmarking procedure
`deploy/README.md`	The Docker development stack and its network layout
`web/README.md`	The admin frontend